

Proyecto 1: WikiSearch

Equipo de trabajo: Granados Retana, Diego - 2022158363 Granados Retana, Daniel - 2022104692 Mora Montes, Diego - 2022104866 Gutierrez Conejo, Eduardo - 2019073558

Bases de Datos II

Documentación

Instrucciones para ejecutar el proyecto

Instrucciones de cómo acceder a la máquina virtual Para acceder a la máquina virtual, primero es necesario acceder al directorio donde se encuentran los archivos del proyecto. Este contiene la llave ssh para conectarse a la máquina virtual. Luego, podemos ejecutar el siguiente comando para conectarnos:

```
ssh -i ssh/db2 ubuntu@{IP pública de la máquina virtual}
```

Ejemplo:

```
ssh -i ssh/db2 ubuntu@207.211.179.27
```

Luego, para poder correr comandos de Docker, debemos elevar el usuario a nivel de root con:

```
sudo -i
```

Ya con esto, podemos ejecutar comandos de Docker para verificar que los 3 componentes están corriendo:

```
docker ps
```

```
root@w01:~# docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS                               NAMES
6db641779f3   dandilego235/wikisearch:latest     "python -u ./api/flat..." About a minute ago Up About a minute 0.0.0.0:5000->5000/tcp, :::5000->5000/tcp, 0.0.0.0:8080->8080/tcp, :::8080->8080/tcp  wikisearch
122386d9de5a   dandilego235/wikisearch:latest     "python -u ./api/flat..." About a minute ago Up About a minute 0.0.0.0:3000->3000/tcp, :::3000->3000/tcp  wikisearch
e43e8e58e4d0   dandilego235/wikisearch:latest     "python -u ./api/flat..." 11 days ago   Up 11 days   0.0.0.0:3000->3000/tcp, :::3000->3000/tcp  wikisearch
```

Cuando ya se logra entrar a la máquina virtual, se obtiene la siguiente página principal.

Login

Get started with **WikiSearch** today!

Login

Sign Up

Uno puede ingresar los datos para ingresar al programa. Estos se validarán con los datos ingresados en Firebase. Si se desea registrar, hay que presionar en el botón de Sign Up. Esto llevará el usuario a la siguiente página.

Register

Get started with **WikiSearch** today!

Email

Password (At least 6 characters)

Phone: (Area Code) Number

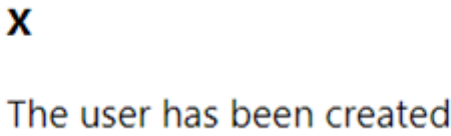
Name

Last Name 1

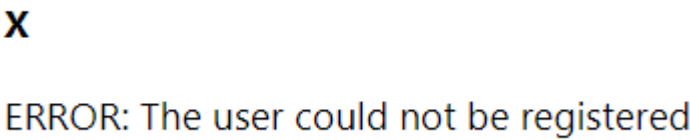
Last Name 2

Register

Aquí, es necesario que la contraseña tenga al menos 6 caracteres y que el número de teléfono tenga el código de area. Por ejemplo, +50612345678. Luego de que se presione el botón de registrar, si el usuario se pudo registrar, se muestra este mensaje de éxito.



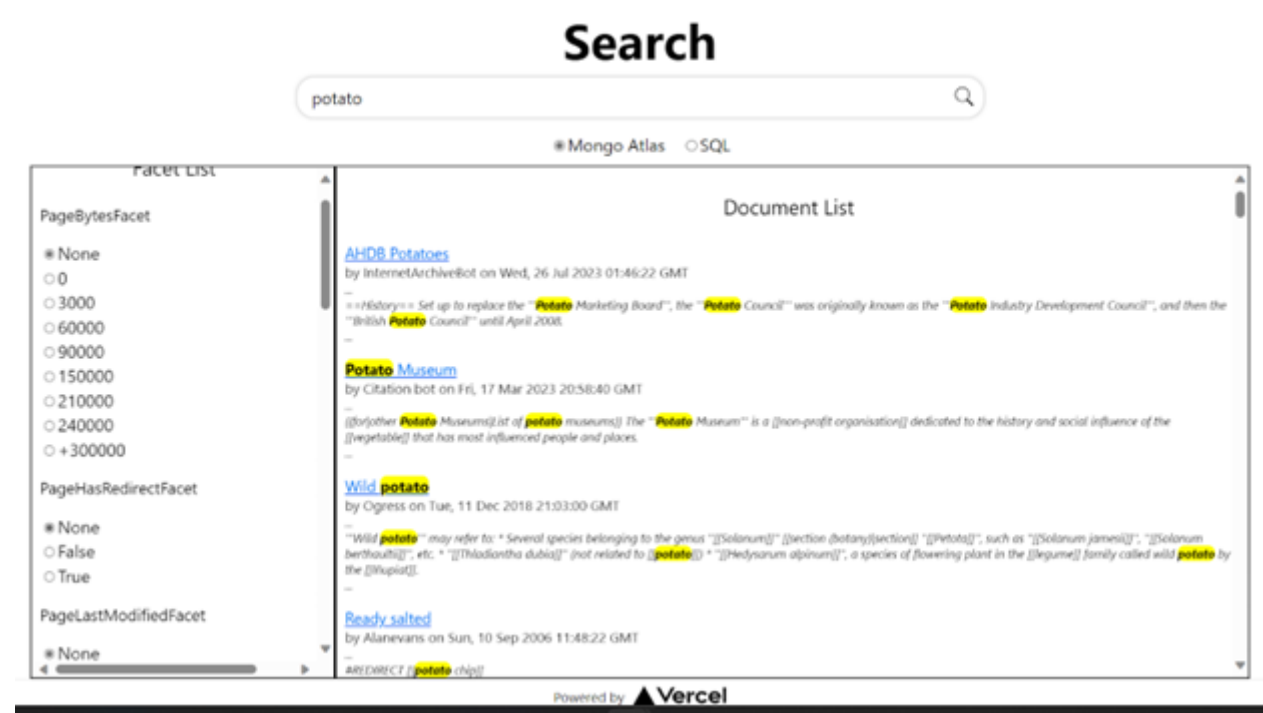
Si no se pudo registrar:



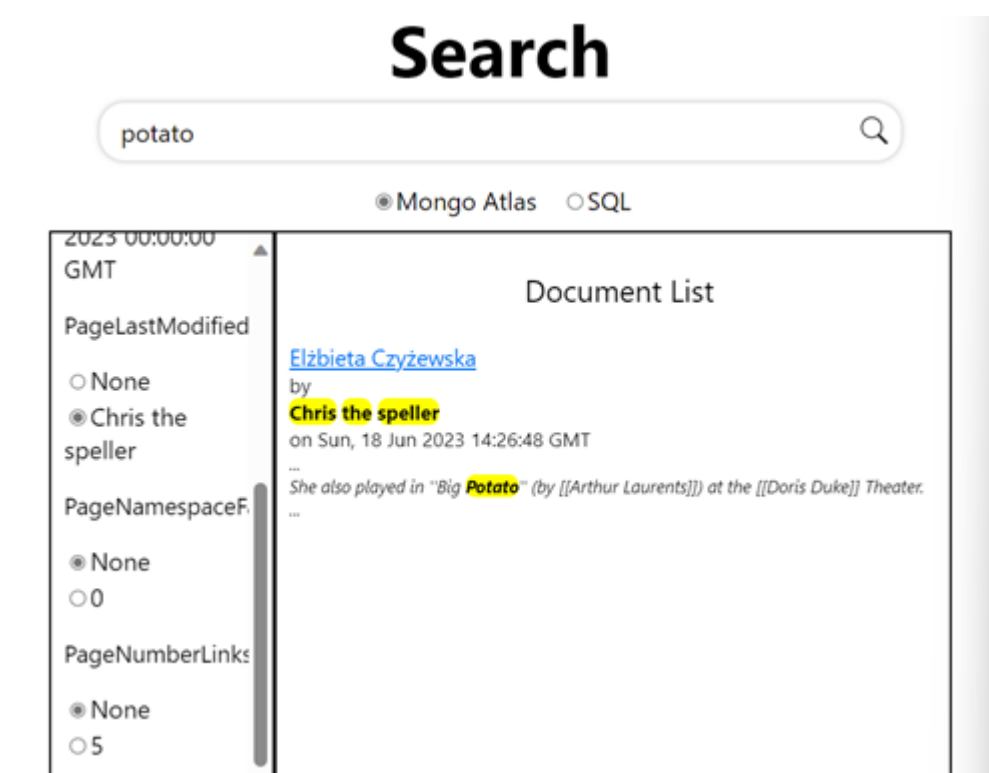
Luego de ingresar al sistema, se llegará a la siguiente pantalla.



Esta es la página para buscar los documentos. Aquí podemos notar que tanto los facets como los documents están vacíos. Esto es porque se llenan dinámicamente con base en lo que uno buscó. En los botones de arriba uno puede seleccionar de cuál base de datos se buscan los documentos. Para esto, se ingresa el valor que se quiere buscar y se presiona en la lupa.



Esta es la página que aparece cuando uno busca algo. En este ejemplo, buscamos potato y estos fueron los facets que se generaron y los documentos que se encontraron. Uno puede seleccionar los facets con los que se quiere buscar y luego volver a presionar en el botón de search para buscar con ese facet.



Por ejemplo, estos son los resultados cuando se busca potato con el facet de Chris the speller. Se pueden presionar en los links del documento, al texto azul subrayado, para ir a una página donde se muestra el documento completo.

Document View

Sotaquirá

2023-06-23T09:27:04 by Marccapelle

Wikipedia Link: <https://en.wikipedia.org/wiki/Sotaquir%C3%A1>

Wikipedia Generated Link: <http://en.wikipedia.org/?curid=6985708>

Document Info:

- Bytes: 4360
- Namespace: 0
- Has Redirect: No information whether it has a redirect available.
- Redirect: No redirect available
- Restrictions: No restrictions available.
- Site Info Name: Wikipedia
- Site Info Data Base Name: enwiki
- Site Language: English

Number of links: 4. Links:

Text

[[Infobox settlement|name = Sotaquirá|native_name = |nickname = |motto = |settlement_type = [[Municipalities of Colombia|Municipality]] and town|image_skyline = |image_size = |image_caption = |image_flag = Flag of Sotaquirá (Boyacá).svg|image_seal = |image_map = Colombia - Boyacá - Sotaquirá.svg|mapsize = 250px|map_caption = Location of the municipality and town of Sotaquirá in the Boyacá Department of Colombia|pushpin_map = |pushpin_mapsize = 300|pushpin_map_caption = Location in Colombia|subdivision_type = Country|subdivision_name = |flag|Colombia]]|subdivision_type1 = |Departments of Colombia|Department]]|subdivision_name1 = |Boyacá Department]]|subdivision_type2 = Province|subdivision_name2 = |Central Boyacá Province]]|leader_title = Mayor|leader_name = Henry Santiago Suta Toca
<small> (2020-2023)</small></small>>|established_title = Founded|established_date = 20 December 1582|founder = [[Friar]] Arturo Cabeza de Vaca|area_magnitude = |area_total_km2 = 288.65|area_total_sq_mi = |area_land_km2 = |area_land_sq_mi = |area_water_km2 = |area_water_sq_mi = |area_water_percent = |area_urban_km2 = 0.10|area_urban_sq_mi = |area_metro_km2 = |area_metro_sq_mi = |population_as_of = 2015|population_note = |population_total = 7709|population_density_km2 = auto|population_density_sq_mi = |population_metro = |population_density_metro_km2 = |population_density_metro_sq_mi = |population_urban = 716|coordinates = |timezone = Colombia Standard Time|utc_offset = -5|timezone_DST = |utc_offset_DST = |elevation_m = 2860|elevation_ft = |website = |http://www.sotaquirá-boyacá.gov.co/|Official website|footnotes = |}}'''Sotaquirá''' is a town and municipality in the [[Central Boyacá Province]], part of the [[Colombia]]|[[Departments of Colombia|Department]] of [[Boyacá Department|Boyacá]]. Sotaquirá is situated in the [[Cordillera Oriental (Colombia)|Eastern Ranges]] of the Colombian [[Andes]] with the small urban center at an altitude of [[convert|2860|m|ft]] at [[convert|29|km|mi]] from [[Tunja]]. Sotaquirá is a highly rural municipality with 90% of the people living in rural areas and about 50% more cows than inhabitants. The municipality borders [[Tuta, Boyacá|Tuta]] in the east, [[Paipa]] in the north, [[Gámbita]], [[Santander Department|Santander]] in the west and [[Cómbita]] in the south.<ref name=websiteSotaquirá>[[in lang|es]] |http://www.sotaquirá-boyacá.gov.co/informacion_general.shtml Official website Sotaquirá</ref> == History == Located on the [[Altiplano Cundiboyacense]], Sotaquirá before the [[Spanish conquest of the Muisca|Spanish conquest]] was inhabited by the [[Muisca people|Muisca]] organized in their loose [[Muisca Confederation]]. Sotaquirá was ruled by the "[[zaque]]" of [[Tunja|Hunza]], present-day department capital Tunja. Modern Sotaquirá was founded on December 20, 1582 by [[Friar]] Arturo Cabeza de Vaca.<ref name=websiteSotaquirá> == Etymology == In the [[Chibcha language]] of the Muisca Sotaquirá means "Town of the sovereign".<ref name=excelsio>[[in lang|es]] |http://www.excelsio.net/2011/11/sabe-el-origen-del-nombre-de-su.html Etymology Sotaquirá - Excelsio.net</ref> == Economy == Main economical activities of Sotaquirá are [[agriculture]] ([[barley]], [[bean]]s, [[potato]]es, [[maize]], [[pea]]s and fruits as [[peach]]es, [[pear]], [[prune]]s, [[apple]]s, [[blackberry|blackberries]], [[strawberry|strawberries]], [[cucurbit]], [[tree tomato]]es and [[feijoa]], [[livestock]] and [[dairy]] farming.<ref name=websiteSotaquirá> == References == {{reflist}} (Municipalities boyacá department)|coord|546|N|731|W|display=title|region:CO_type:city_source:GNS-enwiki]]|Category:Municipalities of Boyacá Department]]|Category:Populated places established in 1582]]|Category:1582 establishments in the Spanish Empire]]

Return to Search

Mongo Atlas y Mongo Atlas Search

Base de datos

La base de datos tiene el nombre de bibliotec, en esta se encuentra una colección donde se van a guardar la información obtenida de los Dumps de Wikipedia. Los documentos tienen los siguientes campos:

- **_id**: El cual es el identificador único para cada documento. Este es un encriptado del nombre de la página de wikipedia. Es de tipo string.
- **PageBytes**: Este representa la cantidad de bytes que posee la página. Es de tipo int.
- **PageHasRedirect**: Indica si la página tiene links de redireccionamiento. Es de tipo string, ya que los valores booleanos no se les puede generar facets ni ser buscados mediante la búsqueda de texto completo.
- **PageId**: Es un identificador único para la página de wikipedia. Es de tipo int.
- **PageLastModified**: Esta es la fecha de la última modificación realizada en la página de wikipedia. Es de tipo date.
- **PageLastModifiedUser**: Este es el último usuario que realizó una modificación sobre la página de Wikipedia. Es de tipo string.
- **PageNamespace**: Este indica el namespace donde se encuentra la página de Wikipedia. Es de tipo string, aunque si valor es numérico, ya que se desea realizar un búsqueda textual sobre este campo.
- **PageRedirect**: En este campo se guarda la redirección que posee la página. Es de tipo string.
- **PageRestrictions**: Es un array con las restricciones que posee la página. Es de tipo array de strings.
- **PageText**: En este campo se encuentra el texto de la página. Es de tipo string.
- **PageTitle**: Representa el título de la página. Es de tipo string.
- **SiteInfoDBName**: Representa el nombre de la base de datos del sitio. Es de tipo string.
- **SiteInfoName**: Representa el nombre del sitio donde se encuentra la información. Es de tipo string.
- **SiteLanguage**: Representa el lenguaje en el que se encuentran los documentos. Es de tipo string.
- **pageWikipediaGenerated**: Representa el link generado de forma automática. Es de tip string.
- **PageLinks**: Es un arreglo con los links asociados a la página. Es de tipo array de strings.
- **PageNumberLinks**: Representa la contidad de links que posee a página. Es de tipo int.

- **PageWikipediaLink:** Es el link donde se encuentra la página. Es de tipo string.

Search Index

La base de datos de bibliotec posee un índice de búsqueda el cual hace posible la búsqueda de texto completo sobre los campos que son de tipo string. Este índice posee el siguiente mapping:

```
{
  "mappings": {
    "dynamic": true,
    "fields": {
      "PageBytes": [
        {
          "indexDoubles": false,
          "representation": "int64",
          "type": "number"
        },
        {
          "type": "string"
        },
        {
          "representation": "int64",
          "type": "numberFacet"
        }
      ],
      "PageHasRedirect": [
        {
          "type": "string"
        },
        {
          "type": "stringFacet"
        }
      ],
      "PageId": [
        {
          "representation": "int64",
          "type": "number"
        },
        {
          "type": "string"
        }
      ],
      "PageLastModified": [
        {
          "type": "string"
        },
        {
          "type": "date"
        },
        {
          "type": "dateFacet"
        }
      ]
    }
  }
}
```

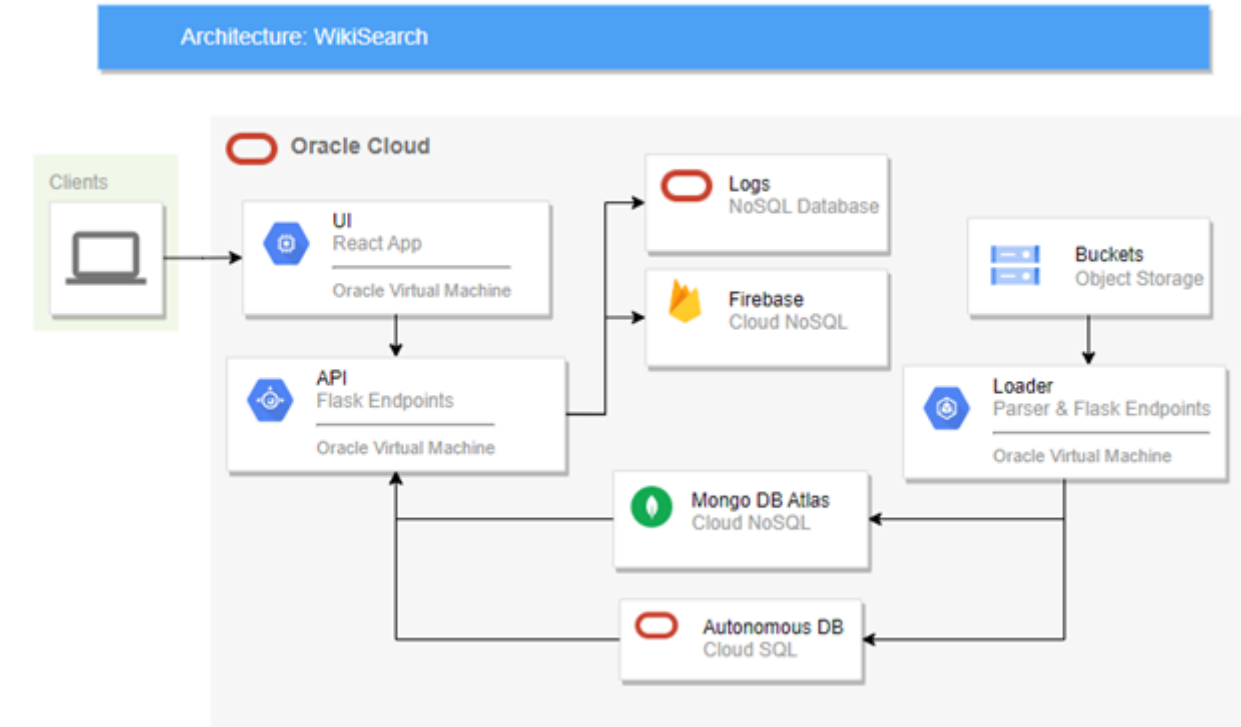
```
],
"PageLastModifiedUser": [
  {
    "type": "string"
  },
  {
    "type": "stringFacet"
  }
],
"PageLinks": {
  "type": "string"
},
"PageNamespace": [
  {
    "type": "string"
  },
  {
    "type": "stringFacet"
  }
],
"PageNumberLinks": [
  {
    "representation": "int64",
    "type": "number"
  },
  {
    "type": "string"
  },
  {
    "representation": "int64",
    "type": "numberFacet"
  }
],
"PageRedirect": {
  "type": "string"
},
"PageRestrictions": [
  {
    "type": "string"
  },
  {
    "type": "stringFacet"
  }
],
"PageText": {
  "type": "string"
},
"PageTitle": {
  "type": "string"
},
"PageWikipediaLink": {
  "type": "string"
},
"SiteInfoDBName": [
```

```
    {
      "type": "string"
    },
    {
      "type": "stringFacet"
    }
  ],
  "SiteInfoName": [
    {
      "type": "string"
    },
    {
      "type": "stringFacet"
    }
  ],
  "SiteLanguage": [
    {
      "type": "string"
    },
    {
      "type": "stringFacet"
    }
  ],
  "pageWikipediaGenerated": {
    "type": "string"
  }
}
}
```

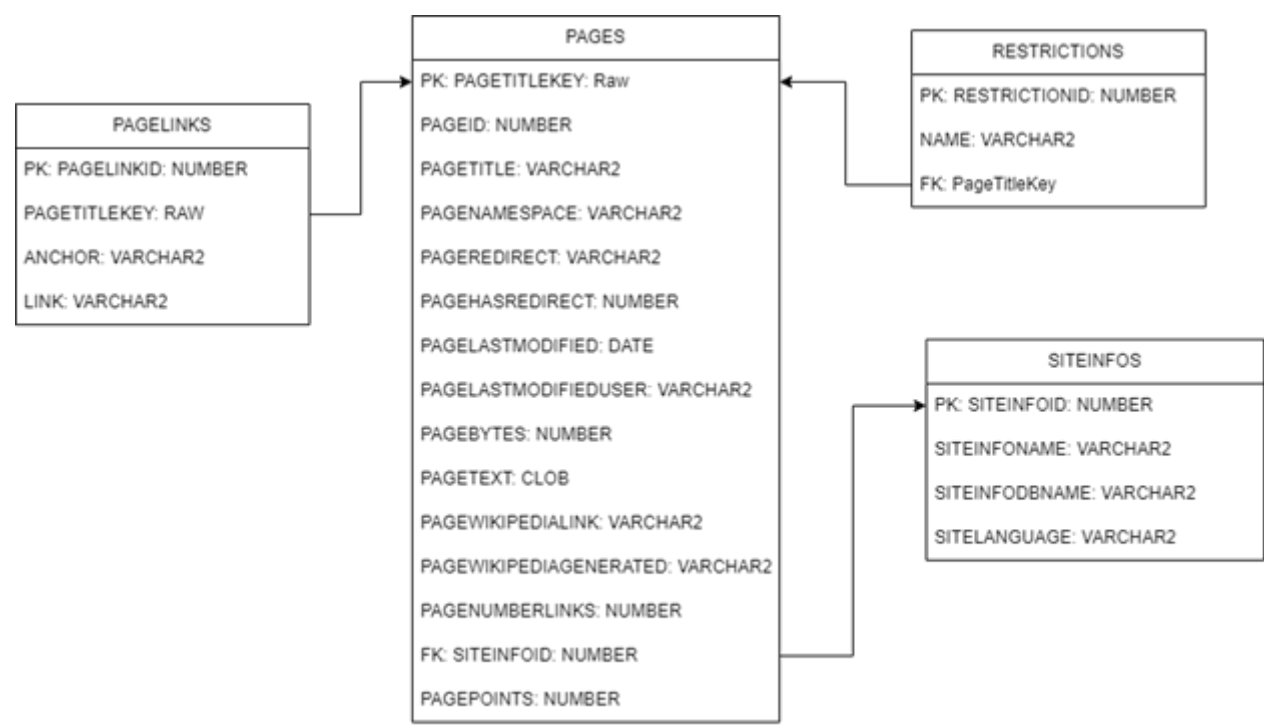
En este índice se definen los tipos de los campos que van a ser utilizados para las búsquedas de texto y los campos que deben de generar facets con buckets para realizar filtros, estos facets pueden ser de tipo string, numéricos, y de fechas. Por ejemplo el campo llamado "SiteLanguage" posee dos tipos, uno de tipo string para realizar las búsquedas, y otro de tipo facet que genera las categorías donde los documentos que retorna la búsqueda puedan ser clasificados.

Diagramas del Proyecto

Arquitectura

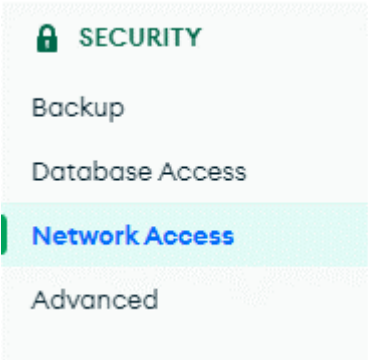


Oracle Autonomous Database



Configuración de IPs

Cabe destacar que el IP de la máquina virtual no se añade automáticamente a la lista de IPs permitidos en Mongo Atlas. Hay que añadirlo manualmente, para hacer esto, hay que ingresar a Mongo Atlas y luego ingresar a la sección de **Network Access** en Security:



Luego, el IP correspondiente a la máquina virtual es el que tiene el comentario de VM en la tabla de IP Access List.

TEC > BIBLIOTEC

Network Access

[IP Access List](#) [Peering](#) [Private Endpoint](#)

[+ ADD IP ADDRESS](#)

You will only be able to connect to your cluster from the following list of IP Addresses:

IP Address	Comment	Status	Actions
201.206.114.3/32	TEC	● Active	EDIT DELETE
186.176.152.202/32 (includes your current IP address)	DANDIEGO	● Active	EDIT DELETE
207.211.179.27/32	VM	● Active	EDIT DELETE
201.198.177.184/32		● Active	EDIT DELETE
181.193.125.1/32	CC	● Active	EDIT DELETE
186.32.199.95/32	DADO	● Active	EDIT DELETE

Para modificarlo, hay que hacer click sobre EDIT y luego pegar el IP público de la máquina virtual donde dice Access List Entry.

Este IP se puede obtener del comando que se ejecuta el comando de creación de la máquina virtual. Igualmente se puede obtener de Oracle Cloud ingresando a la sección de Instances:

Create instances Table settings									
Name	State	Public IP	Private IP	Shape	OCPU count	Memory (GB)	Availability domain	Fault domain	Created
vm01	Running	207.211.179.27	192.168.1.158	Vml Standard E4 Flex	1	6	AD-2	FD-2	Mon, Oct 2, 2023...

Mientras se añade el IP de la máquina virtual a Mongo Atlas, los contenedores del API y el Loader van a estar intentando conectarse a Mongo y van a estar fallando porque no van a tener acceso. Por lo tanto, se implementó la técnica de [retries with exponential backoff](#) para no saturar al servicio. Esta técnica consiste en hacer un intento, si falla, esperar 1 segundo, reintentar, si falla, esperar 2 segundos, luego reintentar, si falla, esperar 4 segundos, y así sucesivamente el tiempo de espera va creciendo de manera exponencial. Esto generalmente se hace por un número limitado de veces y luego se reporta el error, pero como nosotros no queremos que la aplicación se caiga, le pusimos que llegue a esperar un máximo de 256 segundos entre retries. Así, le da suficiente tiempo al usuario de agregar el IP a Mongo Atlas.

```
# Retry with backoff implementado con base en
https://keestalkstech.com/2021/03/python-utility-function-retry-with-exponential-
backoff/#without-typings.
def retry_with_backoff(fn, backoff_in_seconds = 1):
    x = 0
    while True:
        logger.info(x)
        try:
            return fn()
        except:
            # va subiendo de 1, 2, 4, ... hasta esperar 256 segundos entre
            # intentos. Se queda esperando hasta que pueda conectar,
            # porque de lo contrario, no podría trabajar bien.
            sleep = backoff_in_seconds * 2 ** x + random.uniform(0, 1)
            time.sleep(sleep)
            if x < 8:
                x += 1
```

Componentes

Loader

Este componente se encuentra en un contenedor de docker con código en Python dentro de la máquina virtual de Oracle Cloud. Este componente cada 2 minutos escanea el Object Storage y cuando detecta un nuevo archivo realiza el procesamiento de este y luego de procesarlo carga los datos en Mongo Atlas y Autonomous Database, para realizar el procesamiento de los archivos XML se utilizó el módulo `python-mwxml`, el mismo permite obtener de forma sencilla los datos que conforman cada documento. En el Object Storage solo se suben los `enwiki-latest-pages-articles-multistream*.xml-pp.bz2` y los `enwiki-latest-abstract.xml.gz`. El título del artículo encriptado por medio de una función MD5 es la llave primaria en ambas bases de datos.

Código

Parser

El parser es la principal funcionalidad del componente Loader, este se encarga de obtener los campos necesarios de los dumps de wikipedia (archivos de multistream y abstract) y los transforma en datos que son aceptados para insertar en Mongo DB y Autonomous DB. Para este objetivo, se utilizan dos bibliotecas principales [MediaWiki XML Processing](#) y [BigXML](#). La primera de `mwxml` tiene facilita el procesamiento de los archivos multistream del repositorio de archivos de WikiMedia. La segunda facilita el procesamiento de archivos de XML muy grandes, como los abstracts. Usualmente, las bibliotecas suben todo el archivo XML en memoria para trabajarlo. Sin embargo, esto no es posible cuando los archivos son tan grandes como lo son los abstracts. Por lo tanto, la biblioteca `BigXML` permite parsear estos archivos tan grandes cargando solo partes de él a la vez. Esto se logra por medio de la clase *Parser*.

```
# abrimos el archivo para procesarlo con mwxml.
xmlFile = open(f"volume/multistreams/{objectReference.name}", 'rb')
xmlDump = mwxml.Dump.from_file(xmlFile)
```

```
for page in xmlDump.pages:
    pageHasRedirect = 1 if page.redirect else 0
    pageHasRedirect4Mongo = "True" if page.redirect else "False"
    revisions = sorted(page, key=lambda x: x.timestamp,
reverse=True)
    latestRevision = revisions[0]
    latestRevision.timestamp =
datetime.fromtimestamp(latestRevision.timestamp.unix())
    hashkey = hashlib.md5(page.title.encode('UTF-8'))
    pageTitleKey = hashkey.hexdigest()

    # Insert multistream data into Mongo Atlas
    data4MongoMS = [
        page.id,
        page.title,
```

```

        str(page.namespace),
        page.redirect,
        pageHasRedirect4Mongo,
        latestRevision,
        latestRevision.user.text,
        latestRevision.bytes,
        latestRevision.text,
        siteInfo.dbname,
        siteInfo.name,
        "English",
        f"http://en.wikipedia.org/?curid={page.id}",
        page.restrictions
    ]

```

El código anterior es el procesamiento de los datos del archivo de Multistream, donde se obtienen los datos con información relevante y se transforman a los datos que acepta cada una de las bases de datos. Aquí se utiliza la biblioteca de mwxml, específicamente la técnica de [iteración con la clase Dump](#).

```

for item in Parser(abstract).iter_from(Doc):
    url = item.url
    links = item.sublinks
    pageTitle = item.title[11:]
    hashkey = hashlib.md5(pageTitle.encode('UTF-8'))
    pageTitleKey = hashkey.hexdigest()

    # Insert abstract data into Mongo Atlas
    data4MongoA = [
        pageTitle,
        url,
        links,
        len(links)
    ]

```

Este código son los datos que se obtienen del archivo de abstract, donde se obtienen los links y se les da el formato para ingresarlos a la base de datos. Para esta operación se utiliza la biblioteca de BigXML.

MongoDB

```

def upsertDocument(insert_id, values, mongo):
    try:
        db = mongo["bibliotec"]
        query = {"_id": insert_id}
        if len(values) > 3:
            update = {
                "PageId": values[0],

```

```

        "PageTitle": values[1],
        "PageNamespace": values[2],
        "PageRedirect": values[3],
        "PageHasRedirect": values[4],
        "PageLastModified": values[5],
        "PageLastModifiedUser": values[6],
        "PageBytes": values[7],
        "PageText": values[8],
        "SiteInfoDBName": values[9],
        "SiteInfoName": values[10],
        "SiteLanguage": values[11],
        "pageWikipediaGenerated": values[12],
        "PageRestrictions": values[13]}
    else:
        update = {
            "PageWikipediaLink": values[0],
            "PageLinks": values[1],
            "PageNumberLinks": values[2]}

        updateQuery = {"$set": update}

        db["pages"].update_one(query, updateQuery, upsert=True)
except Exception as e:
    raise e

```

El código anterior es la función utilizada en el Loader para cargar los datos a la base de datos de Mongo Atlas. Esta función realiza un upsert, lo que significa que si el campo existe solo lo actualiza, pero si el campo no existe, este se crea y se le asigna el valor especificado. Se realiza de esta forma ya que los archivos que se procesan tienen contenido diferente porque existen documentos donde solo se tienen links, o solo se tiene la información básica, o algunos donde se tiene ambos.

Autonomous DB

Para realizar la inserción a Autonomous Database, se utilizó el siguiente código:

```

# cursor para los operaciones de SQL
cursorSQL = connectionSQL.cursor()

# revisamos si ya hay un siteinfo igual en la base de datos, para
usar ese mismo como FK.
cursorSQL.execute("""SELECT siteInfoId, siteInfoName,
siteInfoDBName FROM siteInfos WHERE siteInfoName = :name AND siteInfoDBName =
:dbname""",
                    [siteInfo.name, siteInfo.dbname])
row = cursorSQL.fetchone()
siteInfoId = cursorSQL.var(int)

# si no hay un siteinfo, lo inserta a la base de datos y guarda el

```

```

id generado en siteInfoId.
    if row is None:
        cursorSQL.execute("""
            INSERT INTO siteInfos (siteInfoName, siteInfoDBName,
siteLanguage) VALUES (:siteInfoName, :siteInfoDBName, 'English')
                                RETURNING siteInfoId INTO :siteInfoId""",
[siteInfo.name, siteInfo.dbname, siteInfoId])
        connectionSQL.commit()
        siteInfoId = siteInfoId.getvalue()[0]
        logger.info("INSERTED SITEINFO")
    # si ya hay un siteInfo con ese nombre y dbname, guarda el
siteInfoId para insertarlo como FK.
    else:
        siteInfoId = row[0]

```

En primer lugar, se abre un cursor para poder realizar las operaciones. Luego se ejecuta el SELECT sobre la tabla de SiteInfos para verificar si ya existe un registro en esta tabla igual a la información del artículo. Esto se realiza para no insertar un Site Info para cada artículo y así ahorrar espacio y evitar duplicación de la información. Si ya existe este registro, se guarda el siteInfoId para usarla como llave foránea en la inserción principal del artículo en el Autonomous Database. Si no existe, se inserta el registro y se usa la cláusula de **RETURNING** para capturar el valor del primary key que se generó en la inserción para luego utilizarlo como llave foránea.

```

        cursorSQL.execute("""
            MERGE INTO pages
            USING (
                SELECT
                    :pageTitleKey as pageTitleKey,
                    :pageId AS pageId,
                    :pageTitle AS pageTitle,
                    :pageNamespace AS pageNamespace,
                    :pageRedirect AS pageRedirect,
                    :pageHasRedirect AS pageHasRedirect,
                    :pageLastModified AS pageLastModified,
                    :pageLastModifiedUser AS pageLastModifiedUser,
                    :pageBytes AS pageBytes,
                    :pageWikipediaLink AS pageWikipediaLink,
                    :pageWikipediaGenerated AS
pageWikipediaGenerated,
                    :pageNumberLinks AS pageNumberLinks,
                    :siteInfoId AS siteInfoId
                FROM dual
            ) pageDump
            ON (pages.pageTitleKey =
HEXTORAW(pageDump.pageTitleKey))
            WHEN MATCHED THEN
                UPDATE SET
                    pages.pageId = pageDump.pageId,
                    pages.pageNamespace = pageDump.pageNamespace,
                    pages.pageRedirect = pageDump.pageRedirect,
                    pages.pageHasRedirect =

```

```

pageDump.pageHasRedirect,
                                pages.pageLastModified =
pageDump.pageLastModified,
                                pages.pageLastModifiedUser =
pageDump.pageLastModifiedUser,
                                pages.pageBytes = pageDump.pageBytes,
                                pages.pageWikipediaGenerated =
pageDump.pageWikipediaGenerated,
                                pages.siteInfoId = pageDump.siteInfoId
                                WHEN NOT MATCHED THEN
                                    INSERT (pageTitleKey, pageId, pageTitle,
pageNamespace, pageRedirect, pageHasRedirect, pageLastModified,
pageLastModifiedUser, pageBytes,
                                pageWikipediaLink, pageWikipediaGenerated,
                                pageNumberLinks, siteInfoId) VALUES
                                (pageDump.pageTitleKey, pageDump.pageId,
pageDump.pageTitle, pageDump.pageNamespace, pageDump.pageRedirect,
pageDump.pageHasRedirect,
                                pageDump.pageLastModified,
                                pageDump.pageLastModifiedUser,
pageDump.pageBytes,
                                pageDump.pageWikipediaLink,
pageDump.pageWikipediaGenerated,
                                pageDump.pageNumberLinks, pageDump.siteInfoId)
                                "",
                                [pageTitleKey, page.id, page.title, page.namespace,
page.redirect, pageHasRedirect, latestRevision.timestamp,
latestRevision.user.text, latestRevision.bytes,
                                None, f"http://en.wikipedia.org/?curid={page.id}",
None, siteInfoId]
                                )
                                # Actualizamos el pageText, ya que este puede ser muy
grande para la tabla DUAL.
                                cursorSQL.execute("""
                                UPDATE pages
                                SET pageText = :pageText
                                WHERE pageTitleKey = HEXTORAW(:pageTitleKey)
                                """, [latestRevision.text, pageTitleKey])
                                if len(page.restrictions):
                                    restrictions =
transformRestrictions(page.restrictions, pageTitleKey)
                                    cursorSQL.executemany(
                                        """
                                        INSERT INTO restrictions (name, pageTitleKey)
                                        VALUES (:name, :pageTitleKey)
                                        """, restrictions
                                    )
                                connectionSQL.commit()

```

En este código, se usó la librería de *oracledb*, la cual permite ejecutar statements de SQL sobre la base de datos. Esta librería utiliza *bind variables* para poder pasarle "parámetros" a estos statements desde el código en Python. Adicionalmente, necesitamos definir un cursor para poder ejecutar statements de [insertar y](#)

actualizar. En este caso, necesitamos insertar el documento si no existe en la base de datos y actualizarlo si ya existe. En Oracle, la instrucción de **Merge** permite realizar una operación similar a esta. Esta permite como “mezclar” tablas. Lo que hace es agarrar dos tablas e intenta mezclar la tabla 2 en la tabla 1 con base en un campo. Si encuentra un “match” entre las tablas, actualiza los campos de la tabla 1 con los de la tabla 2. Si no encuentra el “match”, inserta el registro con la información de la tabla 2. Por lo tanto, puede tener un funcionamiento muy similar al upsert de Mongo. En este caso, queremos agarrar los datos que se leen del dump. Sin embargo, no están en ninguna tabla en la base de datos. Por lo tanto, usamos la tabla dual como tabla temporal para evaluar estas expresiones y utilizarla como la “tabla 2” para hacer el merge. La tabla **dual** usualmente se utiliza para evaluar expresiones. Luego, se inserta en las tablas de PageLinks y de PageRestrictions. Estas deben ser tablas separadas porque un solo artículo puede tener múltiples links o restricciones, por lo que es necesario normalizar estos datos en otras tablas. Como se realizan varias inserciones a la vez, se utiliza la instrucción de oracledb **executemany** para realizar múltiples operaciones a la vez y en conjunto. Al puro final, se realiza la operación de commit en la conexión para guardar los cambios y subirlos a la base de datos.

Estrategia para no repetir procesamiento de archivos.

El contenedor del loader está corriendo con un volumen en la máquina virtual. Este volumen tiene como objetivo descargar los archivos del Object Storage y que se almacenen directamente en la máquina virtual. Esto es porque si en algún momento se tienen que volver a leer, ya están descargados en la máquina virtual y no hay que volverlos a descargar. El mapping de este volumen es:

```
-v "/home/ubuntu/app":/app/volume
```

Durante la ejecución del loader, se crean dos directorios: multistreams y abstracts, donde cada uno almacena sus tipos de archivos respectivos. No obstante, esto es solo una medida provisional, ya que en realidad no se deberían procesar archivos más de una vez, por lo que los archivos descargados del object storage se terminan borrando. La estrategia para no repetir procesamiento de archivos consiste en escribir en un archivo JSON la fecha de creación del último archivo procesado. De esta manera, con cada procesamiento, se va actualizando el archivo con la última fecha de procesamiento. Así, cuando termine de procesar el conjunto de archivos, va a quedarse leyendo el object storage cada 2 minutos e ignorando todos los archivos cuya fecha de creación sea igual o mayor a la fecha que tiene registrada en el archivo de bitácora. Cuando detecta un archivo más nuevo, lo agrega a la cola de procesamiento y lo empieza a procesar. El método para determinar cuál archivo nuevo debe procesar primero se hace con el ordenamiento de los nombres del object storage con base en el time_created. Este ordenamiento debe hacerse para luego descargar los archivos individuales ya que la librería de oci usa la función de *list_objects* para retornar el nombre de los objetos, no el contenido. Este procesamiento se evidencia en el siguiente código:

```
while True:
    try:
        # lee el último timestamp procesado.
        with open("volume/processedLog.json", 'r') as processedLog:
            data = json.load(processedLog)
            lastProcessedTimestamp =
datetime.fromisoformat(data["lastProcessedTimestamp"])
    except Exception as e:
```

```
        logger.error(e)
        tz = pytz.timezone('UTC')
        lastProcessedTimestamp = tz.localize(datetime(1970, 1, 1, 0, 0)) #
        inicia con el valor del epoch

    logger.info("Checking Object Storage...")

    list_objects_response = object_storage.list_objects(namespace,
        bucket_name, fields="timeCreated")
    objectList = sorted(list_objects_response.data.objects, key=lambda x:
        x.time_created, reverse=True)

    objectProcessList = [] # lista donde se guardará el orden de los archivos
    a procesar.

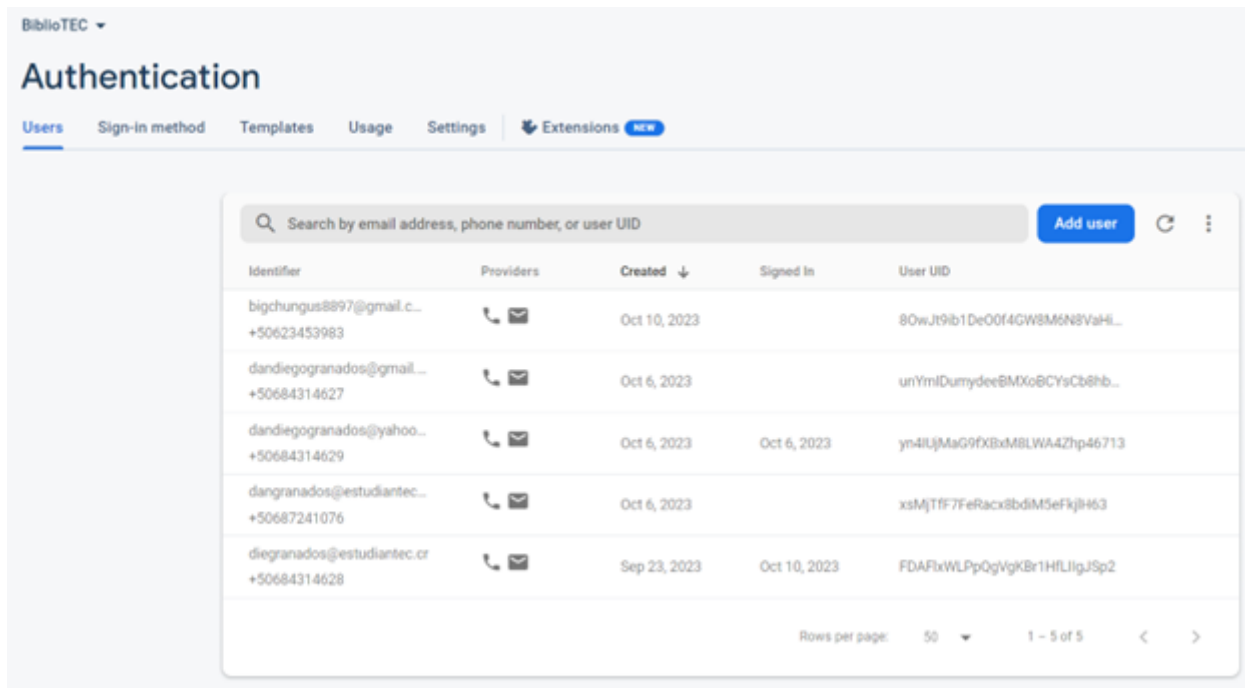
    # recorreremos todos los objetos y agregamos a la lista de iteración solo a
    los que son más nuevos que el último límite de procesamiento
    for objectReference in objectList:
        if objectReference.time_created > lastProcessedTimestamp:
            objectProcessList.insert(0, objectReference)
        else:
            break
```

Ejemplo de la estructura de archivos del volumen dentro de la máquina virtual y del contenido del archivo de bitácora.

```
root@vm01:/home/ubuntu/app# ls
abstracts  multistreams  processedLog.json
root@vm01:/home/ubuntu/app# cat processedLog.json
{"lastProcessedTimestamp": "2023-09-25T20:21:10.820000+00:00"}
```

Firestore

La API es la que se encarga de autenticar al usuario por medio de la base de datos de Firebase Authentication.



Esta es la consola con la que se puede acceder a la información de todos los usuarios (Firebase, s.f.). Podemos ver el identificador, el correo electrónico, el teléfono y el día del registro. En el lado del código, se utilizan dos aspectos de Firebase: El Firebase Admin SDK y el Authentication REST API. Como el API está hecho en Python, se utilizó la librería de Python para esta. Esta librería permite obtener la información del usuario, crear un usuario, actualizarlo, borrarlo y listarlo. En este caso, se utilizó para registrar los usuarios en la base de datos. El código para esto es el siguiente: `user = auth.create_user(email = pEmail, password = pPassword, phone_number = pPhone, display_name = pDisplayName)`

Este código da error si alguna información que se envía no es válida, ya que estas tienen varias restricciones. El teléfono debe adherirse al estándar E.164, por lo que tiene que incluir el código de área. La contraseña debe ser de al menos 6 caracteres. Finalmente, el correo electrónico debe ser válido. No obstante, el Admin SDK no permite verificar la información del usuario con el correo electrónico y la contraseña. Solo existe la función de recuperar un usuario con `get_user_by_email(email)` o `get_user(identificador)`, por lo que no se podría verificar si la contraseña es la correcta. Para esto, se utilizó el Firebase Auth REST API. Esta tiene varias funciones, pero la que se usó es la de sign in con la contraseña y password. Había que realizar un request de POST al siguiente endpoint: `https://identitytoolkit.googleapis.com/v1/accounts:signInWithPassword?key=[API_KEY]` Para obtener el API Key, hay que entrar a la configuración del proyecto:

Your project

Project name

BiblioTEC 

Project ID 

bibliotec-98a06

Project number 

298676919988

Default GCP resource location 

Not yet selected 

Web API Key

AIzaSyAFj0oFcEqOdCL1NF1bGVhvirpxrKqx_LY

Environment

This setting customizes your project for different stages of the app lifecycle

Environment type

Unspecified 

Para realizar el request, se utiliza la librería de requests y el siguiente código:

```
userInfo = json.dumps({"email": email, "password": password,
"return_secure_token":True})
r =
requests.post("https://identitytoolkit.googleapis.com/v1/accounts:signInWithPasswo
rd?key=AIzaSyAFj0oFcEqOdCL1NF1bGVhvirpxrKqx_LY", userInfo)
```

El response de este request es un JSON con la información del usuario, como el identificador y el nombre.

NoSQL Database Table

La API también se encarga de registrar los logs en la tabla de la base de datos NoSQL en Oracle.

Query results do not automatically update when the table is modified. To observe changes, you must run a new query.		
logId	title	bagInfo
1	test	{"id": 3, "text": "test3"}
1695416142	test	{"id": 1, "text": "test"}

La información que contiene la tabla es el logId, el título y la información que se va a almacenar. Para conectarse a la tabla de NoSQL con el API de Python, se necesita obtener varia información de Oracle Cloud. Se necesita obtener un private key, el ocid user, tenancy, region y fingerprint.

OCID: ocid1.user.oc1..aaaaaaaaxmr6fc3rqsoest3yfqamz3yjrulxovyua2
xuwbxb6bdjlyj6lmq [Hide](#) [Copy](#)

API keys

Add API key

Delete

☐	Fingerprint	Created	
☐	aa:4e:83:38:68:4d:38:90:7c:14:b7:84:64:f6:c3:e1	Tue, Sep 19, 2023, 16:55:20 UTC	:
☐	82:51:f2:5f:07:b4:e9:f9:8f:8f:b4:2a:11:e9:95:d4	Tue, Sep 19, 2023, 01:19:52 UTC	:
☐	e6:46:37:90:49:6a:89:88:3d:7b:13:e6:9a:c7:87:c8	Fri, Sep 22, 2023, 18:20:54 UTC	:
0 selected		Showing 3 API keys	< Page 1 >

Tenancy information

Tags

Tenancy information

OCID: ocid1.tenancy.oc1..aaaaaaaab2j6gk2b33sutg2bhoga5zekg3j5su23tygzw6nw5es
4jxdts4ya [Hide](#) [Copy](#)

Name: eduardogc715

Audit retention period: 365 days

Home region: US Midwest (Chicago)

CSI number: 27789369

El código para conectarse es el siguiente. Se utilizan las librerías de oracledb, oci y borneo.

```
config = {
  "key_content": "-----BEGIN PRIVATE KEY-----
MIIIEvQIBADANBgkqhkiG9w0BAQEFAASCBKcwggSjAgEAAoIBAQDM7x0lheMG9Jvn
DUOry1A+XuB6zuCi5nRSiGOqmbQfR/2/20wKteKTbIfRVe6MGGdHdpFTM1ZFZCPd
Vpru6KBk6Aona+iQWXK7tMKfhNj0Tkpj88oeFBPx07mELTQLednaZDaojUhPmWZ6
/tE0BgZAVeu4nyu0KyxmFAL8x2ncXi/BFJgEsRaU00kUns/VfUtFEUWD8BIz1PbR
L/Gi36ZxhqkMPpMs6FwUikYT1eU9KJ4oFgnxX3D4Vi9Fzcg0wrgRFYnNfEOGJZYE
r0LnrqFeowARYAH+nzowjkQ6CSb1QadGq7hzjismxQmxRs56SASKYkrVYVCSBG37
```

```

BMwfYDjrAgMBAAECggEAA8NIBbwFmnUXVRC7SLDnp52FBaboHQBE69cGmB/7sgSt
C2lyu1JHohQAvRPqLgxXU8qWNIQ2y04MEoj/40Rw3X7H4PLBnGo9XxDx7zfj0zaD
dd0zcbCbWnoGvVSPJhQgrzqJKQ3JtscSjnb1tay7pJ6ojMvUZQ8BnZ2R0nsbwK1
62uoZG7XmHaqQJY7z2JZ1a/6Tt6YP2ufaAZxs8tqbfbqKM369LIL/QKJNxEsAncN
/eAiKjyhgPMPfOLxGnS/dWZgPdNjy92+Nw9iYs+5NVqRGiJpiGf4ovWNNPiIG3oL
KbVDhXt3ynwqIS6g6j5ILWicpiUmtstK0eU4oMIOAQKBgQD/sDU/BZ7ljjTQpb/p
YJvRGITpHcXovyH5z/GK6s3gCaJq9/tUihXzaZH5s1V5UOUFVb923KYbUKalyG1F
GRusMF5GWiI55cqFXI3R9KFlu4UJAg1XuCxSWHejweRBV2a4DiV0lj45WuyLeJLR
jsL4CvflVMTZsgVIR0bB/Ti9awKBgQDNLweqq44ksVDGMKWEZIIIZ0PjPUSWhIda
eUMjuazjLxiJ8yiUmfa4AE7vK6SoiP9uudx2xWnnZyVie/PJXseFbINuBh0k9jU6R
H8SOSLFS85TcrCl+ImNDyghiczxaVuiykjYpLIaPqT/BTayXnGiX+xBsweA9Ng1
pZni2fXSGQKBGgFMM8Fy2a+dDEnLj94BDyBSUNqF8KrstLFCpm9DpPIXVy0PoKMQ
L5sSN2Vj7QYD1gVVaxWou3IJSq2wbzPS3o4HUK5EtvJEG/QJv7UFF2RCPN6MShin
NrmBLih5FN2FyrhbXb/KdFY6WB7Cgu+5geLKKRqbUBKEF2sKbd9AmgEjAoGATg93
ZjnwUQtYhJ4bSlwJUrbvx/MWNgFhGD0MCvpyOKw/kKRDl/tP1BCoLbGPdN3m09b
745RT0blRD7NYAmfh9DfUc8LUSyCWHnyiIMlWz6qQq4I9yDUDQU8ZE+dBW2NB+rS
SiXTZ7JnO/52DBQIQthUavgh0bDU1MwU2JY9jIECgYEA4GumexTeBxhwomIKd4bn
5wbYGAGdg6TPym7mGHpEVwVc/SYg1mTeOp988ZIBzdLcjQLY9E5Kpd/dZzVZ5izs
IzI5Mtbaa6QFL3QyhFnyiANbyfuw4rJTgdkUgKlP/jsabVVfMw2x+w5lwPI8oU5E
wf4QTCyd9noRs4piFx6/9A0=
-----END PRIVATE KEY-----""",
    "user":
"ocid1.user.oc1..aaaaaaaxmr6fc3rqsoest3yfqamz3yjrulxovyua2xuwibxb6bdjlyj6lmq",
    "fingerprint": "aa:4e:83:38:68:4d:38:90:7c:14:b7:84:64:f6:c3:e1",
    "tenancy":
"ocid1.tenancy.oc1..aaaaaaaab2j6gk2b33sutg2bhoga5zekg3j5su23tygzw6nw5es4jxdts4ya",
    "region": "us-chicago-1"
}

oci.config.validate_config(config)
object_storage = oci.object_storage.ObjectStorageClient(config)
compartment_id = config['tenancy']
namespace = object_storage.get_namespace().data
bucket_name = "bibliotec"

at_provider = SignatureProvider(tenant_id=config["tenancy"],
                                user_id=config["user"],
                                private_key=config["key_content"],
                                fingerprint=config["fingerprint"])

region = Regions.US_CHICAGO_1

config2 = NoSQLHandleConfig(region, at_provider)

handle = NoSQLHandle(config2)

```

El código para insertar un registro es el siguiente.

```
def write_a_record(handle, table_name, record):
    request = PutRequest().set_table_name(table_name)
    request.set_value(record)
    handle.put(request)
    return
record = {'logId': int(time.time()) + random.randint(0, 30000), 'title': "test",
'bagInfo': json.dumps({"id": 1, "text":"test"})}
write_a_record(handle, 'ic4302_logs', record)
```

Para el logId, se obtiene el tiempo actual y se le suma un número aleatorio. Esto se hace ya que el id no se genera automáticamente, por lo que si se quisiera tener registros únicos con id's consecutivos, habría que leer el id del último, sumarle uno para luego registrar el nuevo log. El bagInfo es la información en formato JSON que va a contener el log.

MongoDB

Para conectarse a Mongo, se utiliza la librería de flask_pymongo. El código de conexión es el siguiente:

```
app.config["MONGO_URI"] =
"mongodb+srv://eduardogc715:BasesII2023@bibliotec.6l341ym.mongodb.net/bibliotec"
mongo = PyMongo(app)
return mongo
```

El Mongo URI se obtiene de la misma página de Mongo Atlas.

Tenancy information

Tags

Tenancy information

OCID: ocid1.tenancy.oc1..aaaaaaaab2j6gk2b33sutg2bhoga5zekg3j5su23tygzw6nw5es4jxdts4ya [Hide](#) [Copy](#)

Name: eduardogc715

Audit retention period: 365 days

Home region: US Midwest (Chicago)

CSI number: 27789369

Para realizar una búsqueda textual completa de un query se debe de utilizar el pipeline de mongo con sus diferentes stages para procesar y recopilar información necesaria de los facets, highlights y los documentos en sí. El siguiente código muestra la creación de dicho pipeline:

El pipeline posee varios stages: "\$search" : En este stage se realiza la búsqueda textual sobre todos los campos que son strings, esto se realiza mediante un operador compuesto donde se define un "should", el

cual es un query el cual la búsqueda debería de cumplir y también se define un "filter", donde se agregan los valores de los facets para realizar los filtros de los documentos. En este mismo stage se guardan los highlights, que de igual manera se indica que sea sobre todos los campos de texto. Finalmente se definen los facets a generar para la búsqueda, estos facets poseen un tipo dependiendo del campo y además para los campos que son facets de tipo numérico y de fecha se deben de definir límites en los cuales caen los diferentes documentos.

"\$facet": Este stage realiza una recopilación de todos los datos obtenidos del stage de "\$search", facets crea nuevos campos: docs: que es donde se encuentran los documentos encontrados de la búsqueda realizada. facets: que es donde se guarda la información obtenida de los facets, contiene los buckets del facet y cantidad de documentos encontrados en cada bucket. Además, dentro de los docs se insertan los metadatos de los highlights, el cual automáticamente incluye solo los highlights de dicho documento y no los de todos.

```
def textSearchQuery(searchQuery):
    pipeline = [
        {
            '$search': {
                'index': 'mainIndex',
                'facet': {
                    'operator': {
                        'compound': {
                            'should': [
                                {
                                    'text': {
                                        'path': {
                                            'wildcard': "*"
                                        },
                                        'query': str(searchQuery)
                                    }
                                }
                            ],
                        },
                    ],
                    'filter': [],
                    'minimumShouldMatch': 1
                }
            },
            'facets': {
                'PageLastModifiedUserFacet': {
                    'type': 'string',
                    'path': 'PageLastModifiedUser'
                },
                'PageNamespaceFacet': {
                    'type': 'string',
                    'path': 'PageNamespace'
                },
                'SiteInfoNameFacet': {
                    'type': 'string',
                    'path': 'SiteInfoName'
                },
                'SiteInfoDBNameFacet': {
                    'type': 'string',
                    'path': 'SiteInfoDBName'
                }
            }
        }
    ]
```



```

    },
    'SiteLanguageFacet': {
      'type': 'string',
      'path': 'SiteLanguage'
    },
    'PageRestrictionsFacet': {
      'type': 'string',
      'path': 'PageRestrictions'
    },
    'PageBytesFacet': {
      'type': 'number',
      'path': 'PageBytes',
      'boundaries': [0, 30000, 60000, 90000, 120000, 150000,
180000, 210000, 240000, 270000, 300000],
      "default": "+300000"
    },
    'PageNumberLinksFacet': {
      'type': 'number',
      'path': 'PageNumberLinks',
      'boundaries': [0, 5, 10, 15, 20, 25, 30, 35, 40, 45, 50],
      "default": "+50"
    },
    "PageLastModifiedFacet": {
      "type": "date",
      "path": "PageLastModified",
      "boundaries": [
        dt.datetime(2023, 1, 1, 0, 0, 0),
        dt.datetime(2023, 2, 1, 0, 0, 0),
        dt.datetime(2023, 3, 1, 0, 0, 0),
        dt.datetime(2023, 4, 1, 0, 0, 0),
        dt.datetime(2023, 5, 1, 0, 0, 0),
        dt.datetime(2023, 6, 1, 0, 0, 0),
        dt.datetime(2023, 7, 1, 0, 0, 0),
        dt.datetime(2023, 8, 1, 0, 0, 0),
        dt.datetime(2023, 9, 1, 0, 0, 0),
        dt.datetime(2023, 10, 1, 0, 0, 0),
        dt.datetime(2023, 11, 1, 0, 0, 0),
        dt.datetime(2023, 12, 1, 0, 0, 0)
      ],
      "default": "Older"
    },
    'PageHasRedirectFacet': {
      'type': 'string',
      'path': 'PageHasRedirect',
    }
  }
},
'highlight': {
  'path': {
    'wildcard': '*'
  },
},
"maxNumPassages": 10000
}

```

```

    }, {
        '$facet': {
            'docs': [
                {
                    "$project": {
                        "_id": {
                            "$toString": "$_id"
                        },
                        "PageBytes": 1,
                        "PageHasRedirect": 1,
                        "PageId": 1,
                        "PageLastModified": 1,
                        "PageLastModifiedUser": 1,
                        "PageLinks": 1,
                        "PageNamespace": 1,
                        "PageNumberLinks": 1,
                        "PageRedirect": 1,
                        "PageRestrictions": 1,
                        "PageText": 1,
                        "PageTitle": 1,
                        "PageWikipediaLink": 1,
                        "SiteInfoDBName": 1,
                        "SiteInfoName": 1,
                        "SiteLanguage": 1,
                        "pageWikipediaGenerated": 1,
                        "highlights": { "$meta": "searchHighlights" }
                    }
                }, {
                    "$limit": 500
                }
            ],
            'facets': [
                {
                    '$replaceWith': '$$SEARCH_META'
                }, {
                    '$limit': 1
                }
            ]
        }
    }
]

if searchQuery.isnumeric():
    pipeline[0]["$search"]["facet"]["operator"]["compound"]
["should"].append({"equals": {"path": "PageBytes", "value": int(searchQuery)}})
    pipeline[0]["$search"]["facet"]["operator"]["compound"]
["should"].append({"equals": {"path": "PageId", "value": int(searchQuery)}})
    pipeline[0]["$search"]["facet"]["operator"]["compound"]
["should"].append({"equals": {"path": "PageNumberLinks", "value":
int(searchQuery)}})
    elif isValidDate(searchQuery):
        pipeline[0]["$search"]["facet"]["operator"]["compound"]
["should"].append({"equals": {"path": "PageLastModified", "value":
dt.datetime.strptime(searchQuery, "%Y-%m-%d")}})
    return pipeline

```

Para agregar los buckets seleccionados y realizar los filtros al pipeline se utiliza una función la cual recibe los filtros en forma de una lista ordenada, el código donde se encuentra es el siguiente:

```
def filteredTextSearchQuery(searchQuery, PageLastModifiedUser, PageNamespace,
SiteInfoName, SiteInfoDBName, SiteLanguage, PageRestrictions, PageBytes,
PageNumberLinks, PageLastModified, PageHasRedirect):
    pipeline = textSearchQuery(searchQuery)
    if PageLastModifiedUser:
        pipeline[0]["$search"]["facet"]["operator"]["compound"]
["filter"].append({"phrase": {"path": "PageLastModifiedUser", "query":
PageLastModifiedUser}})
    if PageNamespace:
        pipeline[0]["$search"]["facet"]["operator"]["compound"]
["filter"].append({"phrase": {"path": "PageNamespace", "query": PageNamespace}})
    if SiteInfoName:
        pipeline[0]["$search"]["facet"]["operator"]["compound"]
["filter"].append({"phrase": {"path": "SiteInfoName", "query": SiteInfoName}})
    if SiteInfoDBName:
        pipeline[0]["$search"]["facet"]["operator"]["compound"]
["filter"].append({"phrase": {"path": "SiteInfoDBName", "query": SiteInfoDBName}})
    if SiteLanguage:
        pipeline[0]["$search"]["facet"]["operator"]["compound"]
["filter"].append({"phrase": {"path": "SiteLanguage", "query": SiteLanguage}})
    if PageRestrictions:
        pipeline[0]["$search"]["facet"]["operator"]["compound"]
["filter"].append({"phrase": {"path": "PageRestrictions", "query":
PageRestrictions}})
    if PageBytes:
        if PageBytes == "+300000":
            pipeline[0]["$search"]["facet"]["operator"]["compound"]
["filter"].append({"range": {"path": "PageBytes", "gte": 300000}})
        else:
            pipeline[0]["$search"]["facet"]["operator"]["compound"]
["filter"].append({"range": {"path": "PageBytes", "lt": (int(PageBytes)+30000),
"gte": int(PageBytes)}})
    if PageNumberLinks:
        if PageNumberLinks == "+50":
            pipeline[0]["$search"]["facet"]["operator"]["compound"]
["filter"].append({"range": {"path": "PageNumberLinks", "gte": 50}})
        else:
            pipeline[0]["$search"]["facet"]["operator"]["compound"]
["filter"].append({"range": {"path": "PageNumberLinks", "lt":
(int(PageNumberLinks)+5), "gte": int(PageNumberLinks)}})
    if PageLastModified:
        if PageLastModified == "Older":
            pipeline[0]["$search"]["facet"]["operator"]["compound"]
["filter"].append({"range": {"path": "PageLastModified", "lte": dt.datetime(2023,
1, 1, 0, 0, 0)}})
        else:
            date_format = "%a, %d %b %Y %H:%M:%S %Z"
            # Parse the date string into a datetime object
```

```

        dateObj = dt.datetime.strptime(PageLastModified, date_format)
        newDate = dateObj + dt.timedelta(days=31)
        pipeline[0]["$search"]["facet"]["operator"]["compound"]
["filter"].append({"range": {"path": "PageLastModified", "lt": newDate, "gte":
dateObj}}})
    if PageHasRedirect:
        pipeline[0]["$search"]["facet"]["operator"]["compound"]
["filter"].append({"phrase": {"path": "PageHasRedirect", "query":
PageHasRedirect}}})
    return pipeline

```

La aplicación API para conectarse a Mongo utiliza 3 endpoints donde realiza las búsquedas y actualizaciones de los datos: `"/mongodb/get_data/<query>"` En este endpoint se define la siguiente función:

```

def get_data (query):
    REQUEST_COUNT.inc()
    filters = request.get_json()
    record = {'logId': int(time.time()) + random.randint(0, 30000), 'title':
"get_data", 'bagInfo': json.dumps({"query": query, "body": filters})}
    write_a_record(handle, 'ic4302_logs', record)
    pipeline = filteredTextSearchQuery(query, filters[0], filters[1], filters[2],
filters[3], filters[4], filters[5], filters[6], filters[7], filters[8],
filters[9])
    results = list(mongo.db.pages.aggregate(pipeline))[0]
    pathsDone = {}
    for doc in results["docs"]:
        for highlight in doc["highlights"]:
            if (highlight["path"] in pathsDone and highlight["score"] >
pathsDone[highlight["path"]]) or highlight["path"] not in pathsDone:
                doc[highlight["path"]] = highlight["texts"]
                pathsDone[highlight["path"]] = highlight["score"]
    return results

```

Esta función obtiene de la base de datos los documentos resultantes de la búsqueda de texto completo con el texto a consultar. Además de esto, esta función maneja los filtros a realizar en la búsqueda, por lo que la búsqueda inicial y el filtrado de los resultados dada la selección de los buckets de los facets lo maneja este endpoint..

`"/mongodb/update_vote/<id>/<vote>"` Este endpoint contiene la siguiente función:

```

def upsertVote(id, vote):
    REQUEST_COUNT.inc()
    record = {'logId': int(time.time()) + random.randint(0, 30000), 'title':
"update_vote", 'bagInfo': json.dumps({"id": id, "vote": vote})}
    write_a_record(handle, 'ic4302_logs', record)
    try:
        query = {"_id": id}

```

```

        voteVal = int(vote)
        update = {'$inc': {'PagePoints': 1}} if voteVal else {'$inc':
{'PagePoint': -1}}
        mongo.db.pages.update_one(query, update, upsert= True)
        return jsonify("Updated the points on the document. ")
    except Exception as e:
        raise e

```

En esta función se recibe el id del documento y el like o dislike que la persona haya dado para calificar el documento encontrado. El endpoint maneja si existe el campo o no de igual forma utilizando un upsert sobre el campo de "PagePoints".

`"/mongodb/get_doc/<id>/<query>"` En este endpoint se encuentra la siguiente función:

```

def get_doc (id, query):
    REQUEST_COUNT.inc()
    record = {'logId': int(time.time()) + random.randint(0, 30000), 'title':
"get_doc", 'bagInfo': json.dumps({"id": id, "query": query})}
    write_a_record(handle, 'ic4302_logs', record)
    pipeline = textSearchQuery(query)
    pipeline[0]["$search"]["facet"]["operator"]["compound"]
["filter"].append({"phrase": {"path": "_id", "query": id}})
    # obtener el documento
    doc = list(mongo.db.pages.aggregate(pipeline))[0]["docs"][0]
    # procesar el documento
    pathsDone = {}
    linkHigh = None
    textHigh = None
    # insertar los highlights en el documento
    for highlight in doc["highlights"]:
        if (highlight["path"] in pathsDone and highlight["score"] >
pathsDone[highlight["path"]]) or highlight["path"] not in pathsDone:
            pathsDone[highlight["path"]] = highlight["score"]
            if isinstance(doc[highlight["path"]], list):
                linkHigh = highlight["texts"]
            elif highlight["path"] != "PageText":
                doc[highlight["path"]] = highlight["texts"]
            elif highlight["path"] == "PageText":
                textHigh = highlight["texts"]

    # incrustar el highlight en el texto completo
    if textHigh != None:
        pageTextHigh = ""
        newPageText = []
        for dictTextHigh in textHigh:
            pageTextHigh += dictTextHigh["value"]
            newPageText.append(dictTextHigh)
        nonHighText = doc["PageText"].split(pageTextHigh)
        doc["PageText"] = []
        doc["PageText"].append({"type": "text", "value": nonHighText[0]})

```

```

doc["PageText"] += newPageText
doc["PageText"].append({"type": "text", "value": nonHighText[1]})

# incrustar el highlight en el texto completo link donde ocurre
pageLinkHigh = ""
newPageLink = []
for dictLinkHigh in linkHigh:
    pageLinkHigh += dictLinkHigh["value"]
    newPageLink.append(dictLinkHigh)
for linkList in doc["PageLinks"]:
    if linkList[0] == pageLinkHigh:
        linkList[0] = newPageLink
return doc

```

En esta función se encarga de realizar la búsqueda de un documento específico y retornarlo de forma completa con los highlights más relevantes en cada campo donde se hayan encontrado estos y los incluye tanto en campos textuales como de listas en los links.

Autonomous DB

Para conectarse a Autonomous Database de Oracle, se utiliza el siguiente código:

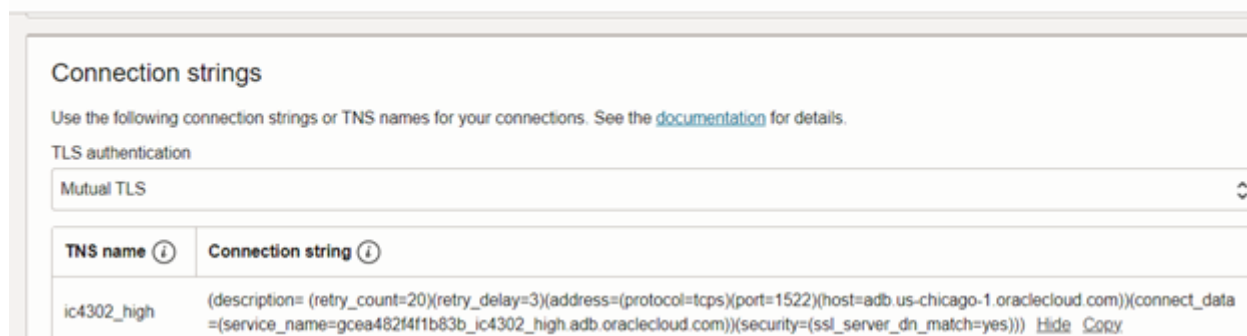
```

cs='''(description= (retry_count=20)(retry_delay=3)(address=(protocol=tcps)
(port=1522)(host=adb.us-chicago-1.oraclecloud.com))(connect_data=
(service_name=gcea482f4f1b83b_ic4302_high.adb.oraclecloud.com))(security=
(ssl_server_dn_match=yes)))'''
connection=oracledb.connect(
    user="ADMIN",
    password="thisiswrongNereo08",
    dsn=cs)

```

El host es la región. El service name se obtiene del mismo Oracle Cloud.

Database connection



Connection strings

Use the following connection strings or TNS names for your connections. See the [documentation](#) for details.

TLS authentication

Mutual TLS

TNS name ⓘ	Connection string ⓘ
ic4302_high	(description= (retry_count=20)(retry_delay=3)(address=(protocol=tcps)(port=1522)(host=adb.us-chicago-1.oraclecloud.com))(connect_data=(service_name=gcea482f4f1b83b_ic4302_high.adb.oraclecloud.com))(security=(ssl_server_dn_match=yes))) Hide Copy

La contraseña se establece en la configuración de la máquina virtual.

```

variable "db_password"{
  description = "Password of the database"
  type        = string
  default     = "thisiswrongNereo08"
}

resource "oci_database_autonomous_database" "autonomous_database" {
  compartment_id = var.compartment_id
  db_name        = var.db_name
  admin_password = var.db_password
  is_free_tier   = true
  is_mtls_connection_required = false
#  tls_authentication = "SERVER"
  whitelisted_ips = [oci_core_instance.vm01.public_ip, "186.176.152.202",
"201.204.89.116", "201.206.114.3", "207.211.176.160", "186.32.199.95"]
}

```

Antes de realizar cualquier búsqueda en Autonomous BD definimos unos índices Full-Text de tipo **CONTEXT INDEX** para las columnas de tipo ya sea VARCHAR2 o CLOB, estos ayudan en búsqueda avanzadas de grandes cantidades de texto para encontrar palabras específicas o frases dentro de nuestros registros, y en el caso de las columnas con tipos de datos NUMERIC o DATE sobre los cuales vamos a hacer búsquedas definimos índices estándar, que nos permiten optimizar las consultas basadas en estos tipos de datos. Estos índices fueron definidos de la siguiente manera:

```

CREATE INDEX idx_pagetitle ON PAGES(PageTitle) INDEXTYPE IS CTXSYS.CONTEXT;
CREATE INDEX idx_pagenamespace ON PAGES(PageNamespace) INDEXTYPE IS
CTXSYS.CONTEXT;
CREATE INDEX idx_pageredirect ON PAGES(PageRedirect) INDEXTYPE IS CTXSYS.CONTEXT;
CREATE INDEX idx_siteinfoname ON SITEINFOS(SiteInfoName) INDEXTYPE IS
CTXSYS.CONTEXT;
CREATE INDEX idx_siteinfodbname ON SITEINFOS(SiteInfoDBName) INDEXTYPE IS
CTXSYS.CONTEXT;
CREATE INDEX idx_sitelanguage ON SITEINFOS(SiteLanguage) INDEXTYPE IS
CTXSYS.CONTEXT;
CREATE INDEX idx_pagewikipedialink ON SITEINFOS(PageWikipediaLink) INDEXTYPE IS
CTXSYS.CONTEXT;
CREATE INDEX idx_pagelastmodifieduser ON PAGES(PageLastModifiedUser) INDEXTYPE IS
CTXSYS.CONTEXT;
CREATE INDEX idx_pagewikipedialink ON PAGES(PageWikipediaLink) INDEXTYPE IS
CTXSYS.CONTEXT;
CREATE INDEX idx_pagewikipediagenerated ON PAGES(pageWikipediaGenerated) INDEXTYPE
IS CTXSYS.CONTEXT;
CREATE INDEX idx_restrictionname ON RESTRICTIONS(Name) INDEXTYPE IS
CTXSYS.CONTEXT;
CREATE INDEX idx_pagelinkanchor ON PAGELINKS(Anchor) INDEXTYPE IS CTXSYS.CONTEXT;
CREATE INDEX idx_pagelinklink ON PAGELINKS(Link) INDEXTYPE IS CTXSYS.CONTEXT;
CREATE INDEX idx_pagetext ON PAGES(PageText) INDEXTYPE IS CTXSYS.CONTEXT;
CREATE INDEX idx_pagebytes ON PAGES(PageBytes);
CREATE INDEX idx_pagehasredirect ON PAGES(PageHasRedirect);

```

```
CREATE INDEX idx_pagelastmodified ON PAGES(PageLastModified);
CREATE INDEX idx_pageid_index ON PAGES(PageId);
```

Ahora sí, para realizar una búsqueda de Autonomous DB se utiliza la conexión previamente creada para definir un cursor de la forma **cursor = connection.cursor()**, y que con este se puedan hacer los distintos llamados a la base de datos. Para el search inicial en donde aún no se tiene ningún facet, creamos una **MATERIALIZED VIEW**, que sería una view indexada en el ambiente de Autonomous. Aunque la creación de esta dure un poco, a la hora de hacer los otros llamados para crear facets, hacer filtraciones con estos u obtener la información de una página específica de los resultados obtenidos, va a ser mucho más eficiente al ser esta una vista indexada con los datos de la consulta actual, esta la definimos mediante el endpoint **/autonomous/get_pages/** (va a ser el mensaje enviado por el usuario que se esté buscando entre las páginas), de la siguiente manera:

```
@app.route('/autonomous/get_pages/<query>', methods=['GET'])
def get_pages(query):

    createAutonomousView(query)
    pages = []
    pages = searchAutonomous()

    return pages
```

En donde createAutonomousView va a ser la funcion en la cual vamos a crear el Materialized View de la siguiente manera:

```
def createAutonomousView(search_term):
    cur = autonomous.cursor()
    try:
        cur.execute('DROP MATERIALIZED VIEW SearchView')
    except:
        pass
    params = [search_term]
    cur.callproc('createSearchView', params)
```

Aquí se intenta de eliminar el materialized view del search pasado y luego mediante el uso del cursor de la coneccion a la base de datos se hace el llamado a **callproc('createSearchView', params)**, el cual es el procedure definido en la base de datos de esta forma:

```
CREATE OR REPLACE PROCEDURE createSearchView( p_search_term IN VARCHAR2)
IS
BEGIN
EXECUTE IMMEDIATE 'CREATE MATERIALIZED VIEW SearchView
BUILD IMMEDIATE
```



```

REFRESH FORCE
ON DEMAND
AS
WITH SearchTerm AS (SELECT '' || p_search_term || '' AS Term FROM DUAL),
LinksAggregate AS(
    SELECT
        l.PAGETITLEKEY, LISTAGG(l.ANCHOR, '', '' ON OVERFLOW TRUNCATE) WITHIN
GROUP (ORDER BY l.ANCHOR) AS LinksList,
        LISTAGG(l.LINK, '', '' ON OVERFLOW TRUNCATE) WITHIN GROUP (ORDER BY
l.LINK) AS LinksLinksList
    FROM
        PAGELINKS l
    GROUP BY
        l.PAGETITLEKEY
),
RestrictionsAggregate AS(
    SELECT
        r.PAGETITLEKEY, LISTAGG(r.Name, '', '' ON OVERFLOW TRUNCATE) WITHIN
GROUP (ORDER BY r.Name) AS RestrictionsList
    FROM
        RESTRICTIONS r
    GROUP BY
        r.PAGETITLEKEY
)
SELECT
    p.PageId,
    p.PageTitle,
    p.PageNamespace,
    p.PageRedirect,
    ra.RestrictionsList,
    p.PageHasRedirect,
    s.SiteInfoName,
    s.SiteInfoDBName,
    s.SiteLanguage,
    p.PageLastModified,
    p.PageLastModifiedUser,
    p.PageBytes,
    p.PageText,
    p.PageWikipediaLink,
    p.pageWikipediaGenerated,
    la.LinksList,
    p.PageNumberLinks,
    p.PagePoints,
    RAWTOHEX(p.PageTitleKey) as PageTitleKey,
    la.LinksLinksList
FROM Pages p
INNER JOIN SITEINFOS s ON p.siteinfoid = s.siteinfoid
CROSS JOIN SearchTerm st
LEFT JOIN LinksAggregate la ON p.PAGETITLEKEY = la.PAGETITLEKEY
LEFT JOIN RestrictionsAggregate ra ON p.PAGETITLEKEY = ra.PAGETITLEKEY
WHERE
    CONTAINS(p.PageTitle, '%' || st.Term || '%') > 0 OR
    CONTAINS(p.PageNamespace, '%' || st.Term || '%') > 0 OR
    CONTAINS(p.PageRedirect, '%' || st.Term || '%') > 0 OR

```

```

CONTAINS(s.SiteInfoName, '%' || st.Term || '%') > 0 OR
CONTAINS(s.SiteInfoDBName, '%' || st.Term || '%') > 0 OR
CONTAINS(s.SiteLanguage, '%' || st.Term || '%') > 0 OR
CONTAINS(p.PageLastModifiedUser, '%' || st.Term || '%') > 0 OR
CONTAINS(p.PageWikipediaLink, '%' || st.Term || '%') > 0 OR
CONTAINS(p.pageWikipediaGenerated, '%' || st.Term || '%') > 0 OR
TO_CHAR(la.LinksList) LIKE '%' || st.Term || '%' OR
TO_CHAR(ra.RestrictionsList) LIKE '%' || st.Term || '%' OR
TO_CHAR(p.PageId) LIKE '%' || st.Term || '%' OR
TO_CHAR(p.PageHasRedirect) LIKE '%' || st.Term || '%' OR
TO_CHAR(p.PageLastModified) LIKE '%' || st.Term || '%' OR
TO_CHAR(p.PageBytes) LIKE '%' || st.Term || '%' OR
TO_CHAR(p.PageNumberLinks) LIKE '%' || st.Term || '%' OR
CONTAINS(p.PageText, '%' || st.Term || '%') > 0 AND
ROWNUM <= 500';
END;

```

En este Procedure estamos creando un Materialized View mediante el llamado de Execute Immediate, en el cual se está haciendo la búsqueda sobre todos los campos requeridos utilizando el mensaje enviado por el usuario, en los campos con un **CONTEXT INDEX** definido utilizamos el llamado **CONTAINS** el cual hace la búsqueda sobre la columna designada del mensaje dado, al mismo tiempo estamos utilizando los wildcards **%** y juntandolos con el mensaje mediante el uso de **||** para que la búsqueda sobre los campos tome en consideración que el texto enviado puede no ser una palabra completa sino una porción de esta, y por último utilizamos múltiples **OR** para de esta forma retornar cualquier página que tenga al menos un match en alguna de las columnas. Ya teniendo esta Materialized View podemos hacer búsquedas sobre esta, y con esto en el mismo endpoint en donde la definimos hacemos una búsqueda de todos los campos de esta y de los facets que se generen con el contenido de esta, estos dos los definimos de la siguiente manera:

```

def searchAutonomous():
    cur = autonomous.cursor()
    cur.execute('SELECT * FROM SearchView')
    result = cur.fetchall()
    pages = []
    for row in result:
        page = {
            'PageId': row[0],
            'PageTitle': row[1],
            'PageNamespace': row[2],
            'PageRedirect': row[3],
            'PageHasRedirect': row[4],
            'PageRestrictions': row[5],
            'SiteInfoName': row[6],
            'SiteInfoDBName': row[7],
            'SiteLanguage': row[8],
            'PageLastModified': row[9].isoformat() if isinstance(row[9],
dt.datetime) else row[9],
            'PageLastModifiedUser': row[10],
            'PageBytes': row[11],
            'PageText': read_lob(row[12]),
            'PageWikipediaLink': row[13],

```

```

        'PageWikipediaGenerated': row[14],
        'PageLinks': row[15],
        'PageNumberLinks': row[16],
        'PagePoints': row[17],
        'PageTitleKey': read_lob(row[18]),
        'PageLinksLinks': row[19]}
    pages.append(page)
cur.close()
facets = []
facets = searchAutonomousFacets()
result = {
    "docs": pages,
    "facets": facets}
return result

```

En esta función hacemos un **SELECT *** de nuestra Materialized View definida mediante el `cursor.execute()`, y almacenamos las páginas recibidas en un diccionario para poder luego accederlas desde el UI, por último en la función hacemos un llamado a la función **searchAutonomousFacets()** en el cual vamos a generar los facets del contenido de la Materialized View, de esta forma:

```

def searchAutonomousFacets():
    cur = autonomous.cursor()
    out_val = cur.var(oracledb.DB_TYPE_CURSOR)
    params = [out_val]
    cur.callproc('search_facets', params)
    result_cursor = out_val.getvalue()
    rows = result_cursor.fetchall()
    result_cursor.close()
    pages = []
    for row in rows:
        page = {
            'facetType': row[0],
            'facetValue': row[1],
            'facetCount': row[2]}
        pages.append(page)
    cur.close()
    return pages

```

En esta función hacemos un llamado con el cursor usando un `callproc()`, como lo habíamos hecho previamente, más en el caso de este procedure enviamos un cursor para poder leer los valores que retorna, dentro de este procedure hacemos selects individuales de las columnas del Materialized View que queramos obtener facets, esto para poder agruparlas sin que haya colisiones entre las columnas y se puedan generar los facets respectivos por cada una, este procedure está definido nuestra Autonomous DB de la siguiente manera:

```

CREATE OR REPLACE PROCEDURE search_facets(facet_cursor OUT SYS_REFCURSOR) IS
BEGIN
    OPEN facet_cursor FOR
    SELECT 'PageNamespaceFacet' AS facet_type, PageNamespace AS id_, COUNT(*) AS

```

```

count
FROM SearchView
WHERE PageNamespace IS NOT NULL
GROUP BY PageNamespace
UNION ALL
SELECT 'PageRestrictionsFacet' , RestrictionsList AS id_, COUNT(*)
FROM SearchView
WHERE RestrictionsList IS NOT NULL
GROUP BY RestrictionsList
UNION ALL
SELECT 'SiteInfoNameFacet', SiteInfoName AS id_, COUNT(*)
FROM SearchView
WHERE SiteInfoName IS NOT NULL
GROUP BY SiteInfoName
UNION ALL
SELECT 'SiteInfoDBNameFacet' , SiteInfoDBName AS id_, COUNT(*)
FROM SearchView
WHERE SiteInfoDBName IS NOT NULL
GROUP BY SiteInfoDBName
UNION ALL
SELECT 'SiteLanguageFacet' , SiteLanguage AS id_, COUNT(*)
FROM SearchView
WHERE SiteLanguage IS NOT NULL
GROUP BY SiteLanguage
UNION ALL
SELECT 'PageLastModifiedUserFacet', PageLastModifiedUser AS id_, COUNT(*)
FROM SearchView
WHERE PageLastModifiedUser IS NOT NULL
GROUP BY PageLastModifiedUser
UNION ALL
SELECT 'PageLastModifiedFacet' AS facet_type, TO_CHAR(EXTRACT(YEAR FROM
PageLastModified)) AS id_, COUNT(*) AS count
FROM SearchView
WHERE PageLastModified IS NOT NULL
GROUP BY EXTRACT(YEAR FROM PageLastModified)
UNION ALL
SELECT 'PageHasRedirectFacet' AS facet_type, TO_CHAR(PageHasRedirect) AS id_,
COUNT(*) AS count
FROM SearchView
WHERE PageHasRedirect IS NOT NULL
GROUP BY PageHasRedirect
UNION ALL
SELECT
'PageBytesFacet' AS facet_type,
CASE
    WHEN PageBytes = 0 THEN '0'
    WHEN PageBytes BETWEEN 1 AND 30000 THEN '1-30000'
    WHEN PageBytes BETWEEN 30001 AND 60000 THEN '30001-60000'
    WHEN PageBytes BETWEEN 60001 AND 90000 THEN '60001-90000'
    WHEN PageBytes BETWEEN 90001 AND 120000 THEN '90001-120000'
    WHEN PageBytes BETWEEN 120001 AND 150000 THEN '120001-150000'
    WHEN PageBytes BETWEEN 150001 AND 180000 THEN '150001-180000'
    WHEN PageBytes BETWEEN 180001 AND 210000 THEN '180001-210000'
    WHEN PageBytes BETWEEN 210001 AND 240000 THEN '210001-240000'

```

```

        WHEN PageBytes BETWEEN 240001 AND 270000 THEN '240001-270000'
        ELSE '+300000'
    END AS id_,
    COUNT(*) AS count
FROM SearchView
WHERE PageBytes IS NOT NULL
GROUP BY
    CASE
        WHEN PageBytes = 0 THEN '0'
        WHEN PageBytes BETWEEN 1 AND 30000 THEN '1-30000'
        WHEN PageBytes BETWEEN 30001 AND 60000 THEN '30001-60000'
        WHEN PageBytes BETWEEN 60001 AND 90000 THEN '60001-90000'
        WHEN PageBytes BETWEEN 90001 AND 120000 THEN '90001-120000'
        WHEN PageBytes BETWEEN 120001 AND 150000 THEN '120001-150000'
        WHEN PageBytes BETWEEN 150001 AND 180000 THEN '150001-180000'
        WHEN PageBytes BETWEEN 180001 AND 210000 THEN '180001-210000'
        WHEN PageBytes BETWEEN 210001 AND 240000 THEN '210001-240000'
        WHEN PageBytes BETWEEN 240001 AND 270000 THEN '240001-270000'
        ELSE '+300000'
    END
UNION ALL
SELECT
'PageNumberLinksFacet' AS facet_type,
    CASE
        WHEN PageNumberLinks = 0 THEN '0'
        WHEN PageNumberLinks BETWEEN 1 AND 5 THEN '1-5'
        WHEN PageNumberLinks BETWEEN 6 AND 10 THEN '6-10'
        WHEN PageNumberLinks BETWEEN 11 AND 15 THEN '11-15'
        WHEN PageNumberLinks BETWEEN 16 AND 20 THEN '16-20'
        WHEN PageNumberLinks BETWEEN 21 AND 25 THEN '21-25'
        WHEN PageNumberLinks BETWEEN 26 AND 30 THEN '26-30'
        WHEN PageNumberLinks BETWEEN 31 AND 35 THEN '31-35'
        WHEN PageNumberLinks BETWEEN 36 AND 40 THEN '36-40'
        WHEN PageNumberLinks BETWEEN 41 AND 45 THEN '41-45'
        ELSE '+50'
    END AS id_,
    COUNT(*) AS count
FROM SearchView
WHERE PageNumberLinks IS NOT NULL
GROUP BY
    CASE
        WHEN PageNumberLinks = 0 THEN '0'
        WHEN PageNumberLinks BETWEEN 1 AND 5 THEN '1-5'
        WHEN PageNumberLinks BETWEEN 6 AND 10 THEN '6-10'
        WHEN PageNumberLinks BETWEEN 11 AND 15 THEN '11-15'
        WHEN PageNumberLinks BETWEEN 16 AND 20 THEN '16-20'
        WHEN PageNumberLinks BETWEEN 21 AND 25 THEN '21-25'
        WHEN PageNumberLinks BETWEEN 26 AND 30 THEN '26-30'
        WHEN PageNumberLinks BETWEEN 31 AND 35 THEN '31-35'
        WHEN PageNumberLinks BETWEEN 36 AND 40 THEN '36-40'
        WHEN PageNumberLinks BETWEEN 41 AND 45 THEN '41-45'
        ELSE '+50'
    END;
END search_facets;

```

En donde utilizamos múltiples **UNION ALL** para retornar una sola tabla con todos los resultados juntos, y por esto es que aseguramos que los campos de cada uno de estos select tengan el mismo label para así juntarlos sin problema. Ya obteniendo los resultados de este procedure y habiendolos retornado a la función **SearchAutonomousFacets()**, los agregamos en un diccionario el cual retornamos a la función **SearchAutonomous()**, en la cual agarramos el diccionario con las páginas y el diccionario con los facets y los juntamos en otro diccionario, por lo que retornamos este último diccionario a la función de nuestro endpoint de Flask y lo devolvemos en el return. Ya teniendo resultados de las páginas y de los facets podemos hacer otras búsquedas en el API, como hacer un search de las páginas obtenidas con facets, esto lo hacemos mediante el uso del endpoint **/autonomous/get_pages_facets/** en el cual se obtiene una lista con diez strings los cuales van a ser utilizados como los diez facets sobre los cuales se van a hacer la búsqueda, este endpoint se ve de la siguiente manera:

```
@app.route('/autonomous/get_pages_facets/', methods=['POST'])
def get_pages_facets():
    REQUEST_COUNT.inc()
    filters = request.get_json()
    print(filters)
    search = searchAutonomousWithFacets(filters[0], filters[1], filters[2],
    filters[3], filters[4], filters[5], filters[6], filters[7], filters[8],
    filters[9])
    return search
```

En donde utilizando los facets obtenidos hacemos un llamado a la función **searchAutonomousWithFacets()**, en la cual definimos un puntero de salida para obtener los resultados y extraemos los valores de mínimo y máximo de los facets de PageBytes y PageNumberLinks de ser necesario para así poder hacer los filtros dentro del llamado de Autonomous DB, esta función se ve de la siguiente manera:

```
def searchAutonomousWithFacets( facet0 ,facet1, facet2, facet3, facet4, facet5,
facet6, facet7, facet8, facet9):
    cur = autonomous.cursor()

    out_val = cur.var(oracledb.DB_TYPE_CURSOR)
    max_bytes = ""
    min_bytes = ""
    min_links = ""
    max_links = ""
    if facet6 != "":
        if facet6 == '0':
            max_bytes = '0'
            min_bytes = '0'
        elif facet6 == '+300000':
            min_bytes = '270001'
            max_bytes = '1000000000000000'
        else:
            min_bytes, max_bytes = map(str, facet6.split('-'))
    if facet7 != "":
```

```
        if facet7 == '0':
            min_links = '0'
            max_links = '0'
        elif facet7 == '+50':
            min_links = '46'
            max_links = '1000000000000000'
        else:
            min_links, max_links = map(str, facet7.split('-'))
        params = [facet0 ,facet1, facet2, facet3, facet4, facet5, min_bytes,
max_bytes, min_links, max_links, facet8, facet9, out_val]

        cur.callproc('searchWithFacets', params)

        result_cursor = out_val.getvalue()
        rows = result_cursor.fetchall()

        result_cursor.close()

        pages = []
        for row in rows:
            page = {
                'PageId': row[0],
                'PageTitle': row[1],
                'PageNamespace': row[2],
                'PageRedirect': row[3],
                'PageHasRedirect': row[4],
                'PageRestrictions': row[5],
                'SiteInfoName': row[6],
                'SiteInfoDBName': row[7],
                'SiteLanguage': row[8],
                'PageLastModified': row[9].isoformat() if isinstance(row[9],
dt.datetime) else row[9],
                'PageLastModifiedUser': row[10],
                'PageBytes': row[11],
                'PageText': read_lob(row[12]),
                'PageWikipediaLink': row[13],
                'PageWikipediaGenerated': row[14],
                'PageLinks': row[15],
                'PageNumberLinks': row[16],
                'PagePoints': row[17],
                'PageTitleKey': read_lob(row[18]),
                'PageLinksLinks': row[19]}
            pages.append(page)

        cur.close()
        result = {
            "docs": pages,
            "facets": "123"}
        return result
```

Dentro de la función una vez que tengamos todos los parámetros hacemos el llamado al procedure de Autonomous DB **SearchWithFacets**, en el cual hacemos un select de los valores del Materialized View y hacemos los filtros respectivos en el caso de que exista un facet para esa columna, este procedure está definido de esta manera:

```
CREATE OR REPLACE PROCEDURE searchWithFacets(
    p_PageLastModifiedUser      IN VARCHAR2 DEFAULT NULL,
    p_PageNamespace             IN VARCHAR2 DEFAULT NULL,
    p_SiteInfoName              IN VARCHAR2 DEFAULT NULL,
    p_SiteInfoDBName            IN VARCHAR2 DEFAULT NULL,
    p_SiteLanguage              IN VARCHAR2 DEFAULT NULL,
    p_pageRestrictions           IN VARCHAR2 DEFAULT NULL,
    p_PageBytesMin              IN VARCHAR2 DEFAULT NULL,
    p_PageBytesMax              IN VARCHAR2 DEFAULT NULL,
    p_pageNumberLinksMin        IN VARCHAR2 DEFAULT NULL,
    p_pageNumberLinksMax        IN VARCHAR2 DEFAULT NULL,
    p_PageLastModified          IN VARCHAR2 DEFAULT NULL,
    p_PageHasRedirect            IN VARCHAR2 DEFAULT NULL,
    main_cursor                  OUT SYS_REFCURSOR
) IS
BEGIN
    OPEN main_cursor FOR
    SELECT *
    FROM SearchView sv
    WHERE
        (p_PageLastModifiedUser IS NULL OR sv.PageLastModifiedUser LIKE '%' ||
p_PageLastModifiedUser || '%')
        AND (p_PageNamespace IS NULL OR sv.PageNamespace LIKE '%' || p_PageNamespace
|| '%')
        AND (p_SiteInfoName IS NULL OR sv.SiteInfoName LIKE '%' || p_SiteInfoName ||
'%')
        AND (p_SiteInfoDBName IS NULL OR sv.SiteInfoDBName LIKE '%' ||
p_SiteInfoDBName || '%')
        AND (p_SiteLanguage IS NULL OR sv.SiteLanguage LIKE '%' || p_SiteLanguage ||
'%')
        AND (p_pageRestrictions IS NULL OR sv.RestrictionsList LIKE '%' ||
p_pageRestrictions || '%')
        AND ((p_PageBytesMin IS NULL OR p_PageBytesMax IS NULL) OR (sv.PageBytes
BETWEEN TO_NUMBER(p_PageBytesMin) AND TO_NUMBER(p_PageBytesMax)))
        AND ((p_pageNumberLinksMin IS NULL OR p_pageNumberLinksMax IS NULL) OR
(sv.PAGENUMBERLINKS BETWEEN TO_NUMBER(p_pageNumberLinksMin) AND
TO_NUMBER(p_pageNumberLinksMax)))
        AND (p_PageLastModified IS NULL OR TO_CHAR(EXTRACT(YEAR FROM
sv.PageLastModified)) = p_PageLastModified)
        AND (p_PageHasRedirect IS NULL OR sv.PageHasRedirect LIKE '%' ||
p_PageHasRedirect || '%');
END searchWithFacets;
```


El cual retorna las páginas que cumplan los filtros, las cuales agregamos en un diccionario el cual retornamos en el endpoint de Flask para así luego poder leer y procesar en el UI. Y en el caso del api de Autonomous DB tenemos dos últimos endpoints, el primero lo utilizamos en el UI en el momento en el que queramos hacer un voto por una página ya sea a favor o en contra este va a llamar al procedure **update_pagepoints()** el cual va a recibir el **PageTitleKey** de la página por la que se esté votando y ya sea un 1 o -1 dependiendo del voto y hace un update de la página respectiva sumándole el valor dado a PagePoints. Endpoint de Flask:

```
@app.route('/autonomous/update_pagepoints/<pageId>', methods=['PUT'])
def update_pagepoints(pageId):
    value = request.json['value']
    cur = autonomous.cursor()
    params = [pageId, value]
    cur.callproc('update_pagepoints', params)
    points = getAutonomousPoints(pageId)
    return str(points)
```

Procedure:

```
CREATE OR REPLACE PROCEDURE update_pagepoints(p_PageId NUMBER, p_Value NUMBER) IS
BEGIN
    UPDATE Pages
    SET PagePoints = PagePoints + p_Value
    WHERE PageId = p_PageId;
    COMMIT;
END update_pagepoints;
```

Y por último tenemos el endpoint **/autonomous/get_page/** en el cual se hace la búsqueda de una pagina especifica en la base de datos, recibiendo el **PageTitleKey** de la página que se quiera obtener y se hace un llamado a la función **autonomousGetPage()** en la cual se hace un **SELECT * FROM SearchView WHERE PageTitleKey = id**, para así obtener la página correspondiente la cual luego metemos en un diccionario para poder retornar y ser leída en el UI. Endpoint:

```
@app.route('/autonomous/get_page/<id>', methods=['POST'])
def get_page(id):

    search = autonomousGetPage(id)
    app.logger.debug(search)
    return search
```

Funcion autonomousGetPage():

```
def autonomousGetPage(id):
    cur = autonomous.cursor()
    cur.execute('SELECT * FROM SearchView WHERE PageTitleKey = :id', id=id)
    result = cur.fetchall()
```

```
pages = []
for row in result:
    page = {
        'PageId': row[0],
        'PageTitle': row[1],
        'PageNamespace': row[2],
        'PageRedirect': row[3],
        'PageHasRedirect': row[4],
        'PageRestrictions': row[5],
        'SiteInfoName': row[6],
        'SiteInfoDBName': row[7],
        'SiteLanguage': row[8],
        'PageLastModified': row[9].isoformat() if isinstance(row[9],
dt.datetime) else row[9],
        'PageLastModifiedUser': row[10],
        'PageBytes': row[11],
        'PageText': read_lob(row[12]),
        'PageWikipediaLink': row[13],
        'pageWikipediaGenerated': row[14],
        'PageLinks': row[15],
        'PageNumberLinks': row[16],
        'PagePoints': row[17],
        'PageTitleKey': read_lob(row[18]),
        'PageLinksLinks': row[19]}
    pages.append(page)
cur.close()
return pages
```

Unit Tests

Pruebas realizadas y pasos para reproducirlas

Para probar los endpoints del API, utilizamos Thunder Client. Esta es una extensión en Visual Studio Code que envía requests HTTP a un endpoint que uno le indique. Uno puede configurar los headers y el cuerpo, el cual puede ser un JSON, XML, Text, Form, Form-encoded, GraphQL y Binary. Por ejemplo, en el caso del login y register, los datos se envían por medio de JSON.

Para probar el loader, se creó un unit test que revisa las funciones de transformación para insertar en las tablas auxiliares de Autonomous Database. Para correrla, simplemente hay que correr el archivo test_loader.py. Este también se debería cada vez que se crea un contenedor de Docker con el loader.

Resultados de las pruebas unitarias

Resultado de la prueba del loader: ..

OK

GET
http://localhost:8000
Send

Status: 200 OK
Size: 5.62 KB
Time: 92 ms

Query
Headers
Auth
Body
Tests
Pre Run

Query Parameters

☐ parameter value

Response
Headers
Cookies
Results
Docs

```

1 # HELP python_gc_objects_collected_total Objects collected during gc
2 # TYPE python_gc_objects_collected_total counter
3 python_gc_objects_collected_total{generation="0"} 622.0
4 python_gc_objects_collected_total{generation="1"} 338.0
5 python_gc_objects_collected_total{generation="2"} 0.0
6 # HELP python_gc_objects_uncollectable_total Uncollectable objects found during GC
7 # TYPE python_gc_objects_uncollectable_total counter
8 python_gc_objects_uncollectable_total{generation="0"} 0.0
9 python_gc_objects_uncollectable_total{generation="1"} 0.0
10 python_gc_objects_uncollectable_total{generation="2"} 0.0
11 # HELP python_gc_collections_total Number of times this generation was collected
12 # TYPE python_gc_collections_total counter
13 python_gc_collections_total{generation="0"} 180.0
14 python_gc_collections_total{generation="1"} 17.0
15 python_gc_collections_total{generation="2"} 1.0
16 # HELP python_info Python platform information
17 # TYPE python_info gauge
18 python_info{implementation="CPython",major="3",minor="11",patchlevel="4",version="3.11.4"} 1.0
19 # HELP flask_http_requests_total Number of HTTP requests received
20 # TYPE flask_http_requests_total counter
21 flask_http_requests_total 1.0
22 # HELP flask_http_requests_created Number of HTTP requests received
23 # TYPE flask_http_requests_created gauge
24 flask_http_requests_created 1.607171864188905e+09
25 # HELP flask_http_requests_autonomous_total Number of HTTP requests received
26 # TYPE flask_http_requests_autonomous_total counter
27 flask_http_requests_autonomous_total 0.0
28 # HELP flask_http_requests_autonomous_created Number of HTTP requests received
29 # TYPE flask_http_requests_autonomous_created gauge
30 flask_http_requests_autonomous_created 1.607171864188905e+09
31 # HELP request_processing_seconds_max_autonomous Maximum request processing time in autonomous endpoint
32 # TYPE request_processing_seconds_max_autonomous gauge
33 request_processing_seconds_max_autonomous 0.0
34 # HELP request_processing_seconds_min_autonomous Minimum request processing time in autonomous endpoint
35 # TYPE request_processing_seconds_min_autonomous gauge
36 request_processing_seconds_min_autonomous 0.0
37 # HELP request_processing_seconds_avg_autonomous Average request processing time in autonomous endpoint
38 # TYPE request_processing_seconds_avg_autonomous gauge
39 request_processing_seconds_avg_autonomous 0.0
40 # HELP flask_http_requests_songs_total Number of HTTP requests received in mongo endpoint

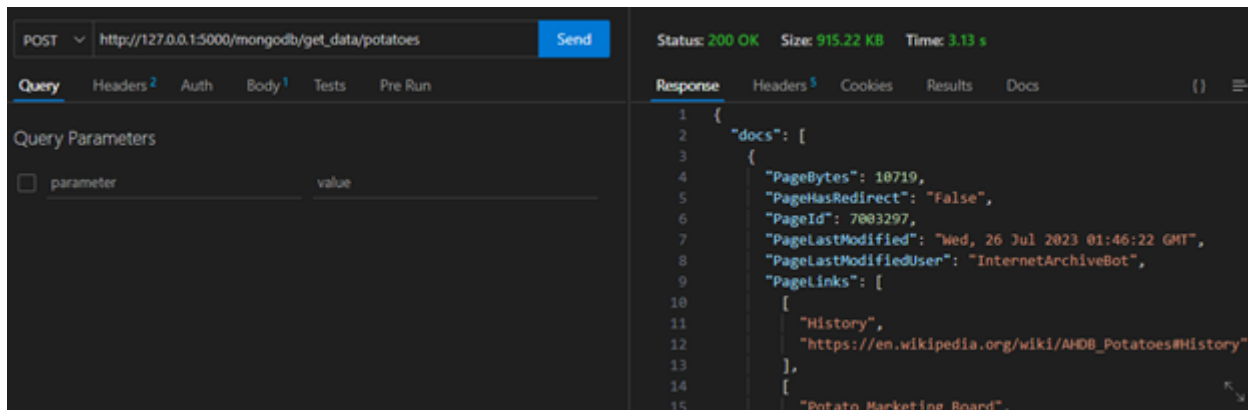
```

43 / 50

Mongo DB

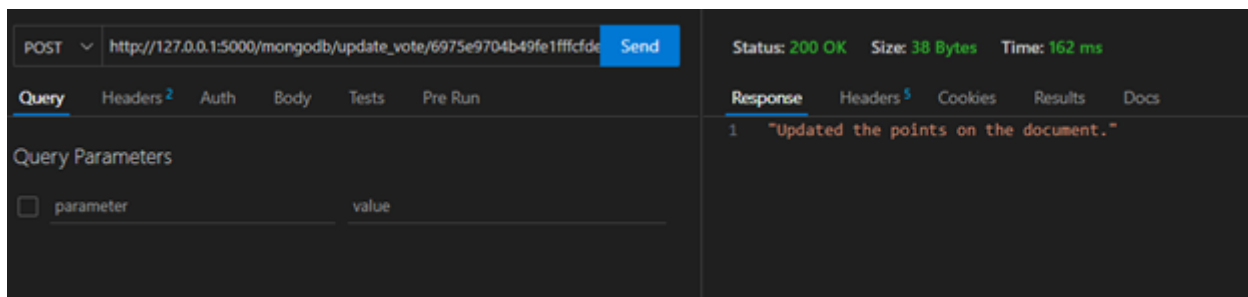
Para cada uno de los endpoint se realizan pruebas de unit testing para validar los resultados obtenidos en cada uno:

`"/mongodb/get_data/<query>"`



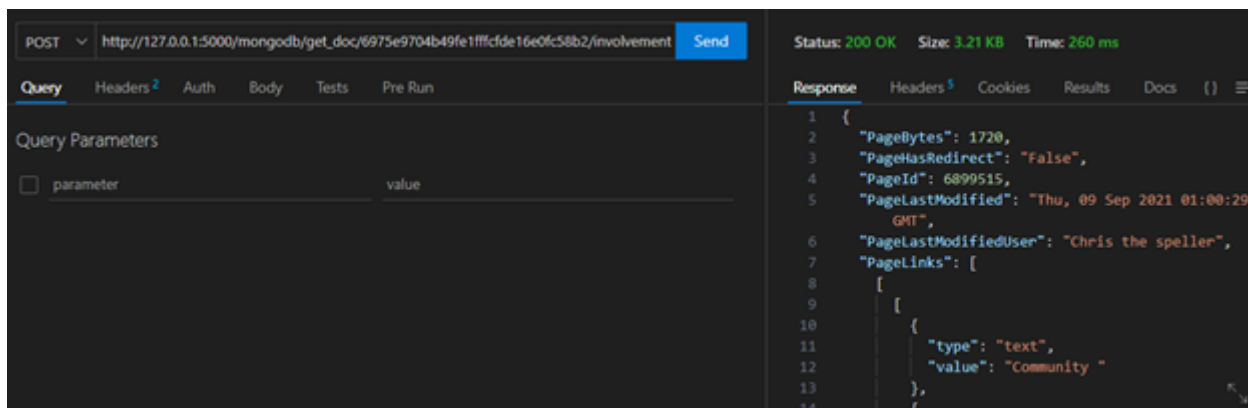
El resultado esperado de este endpoint es un diccionario con dos llaves, un docs, donde se encuentran los documentos que retorna la búsqueda y facets, que es donde se encuentra la información de los buckets y la cantidad de documentos que caen en cada uno de ellos.

`"/mongodb/update_vote/<id>/<vote>"`



El resultado esperado de este endpoint es que retorne el mensaje de que los puntos en el documento han sido actualizados, cuando se realiza esta operación se puede consultar el documento para verificar que los puntos han aumentado o disminuido.

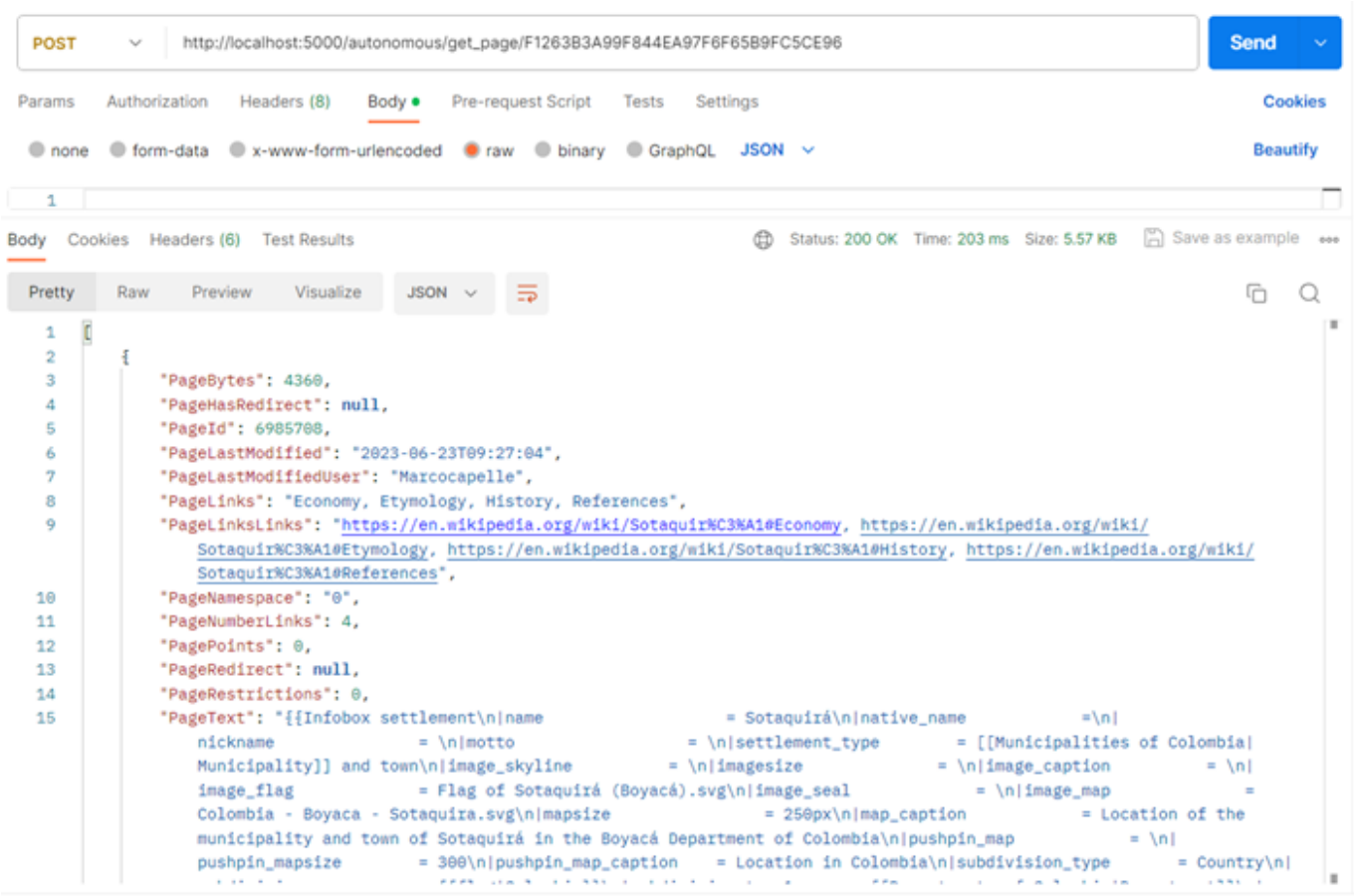
`"/mongodb/get_doc/<id>/<query>"`



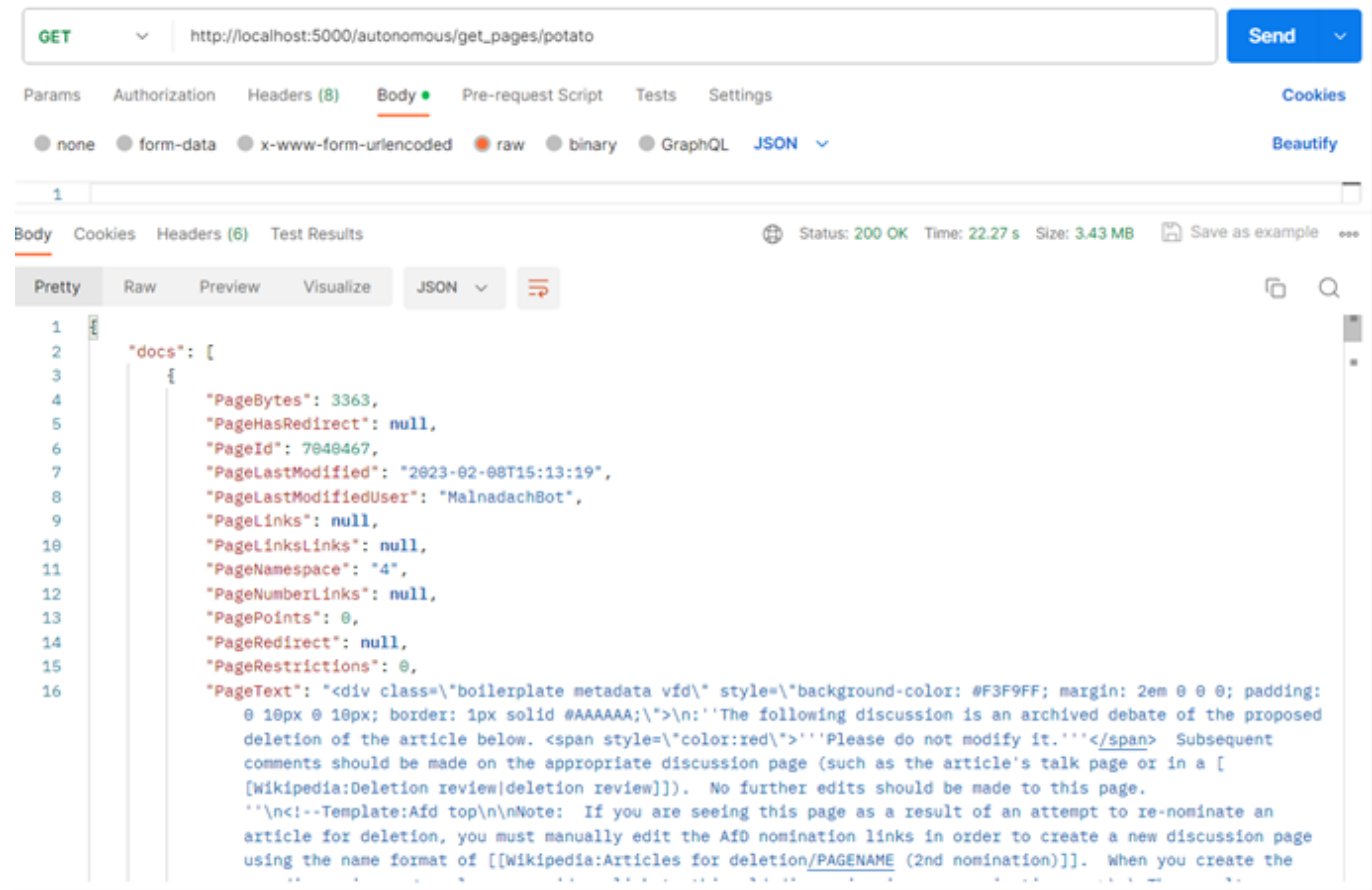
Para este endpoint se espera que se retorne solo el documento solicitado y dentro de él se encuentren los highlights incrustados listos para ser procesados por el UI para realizar el highlight gráfico.

Autonomous DB

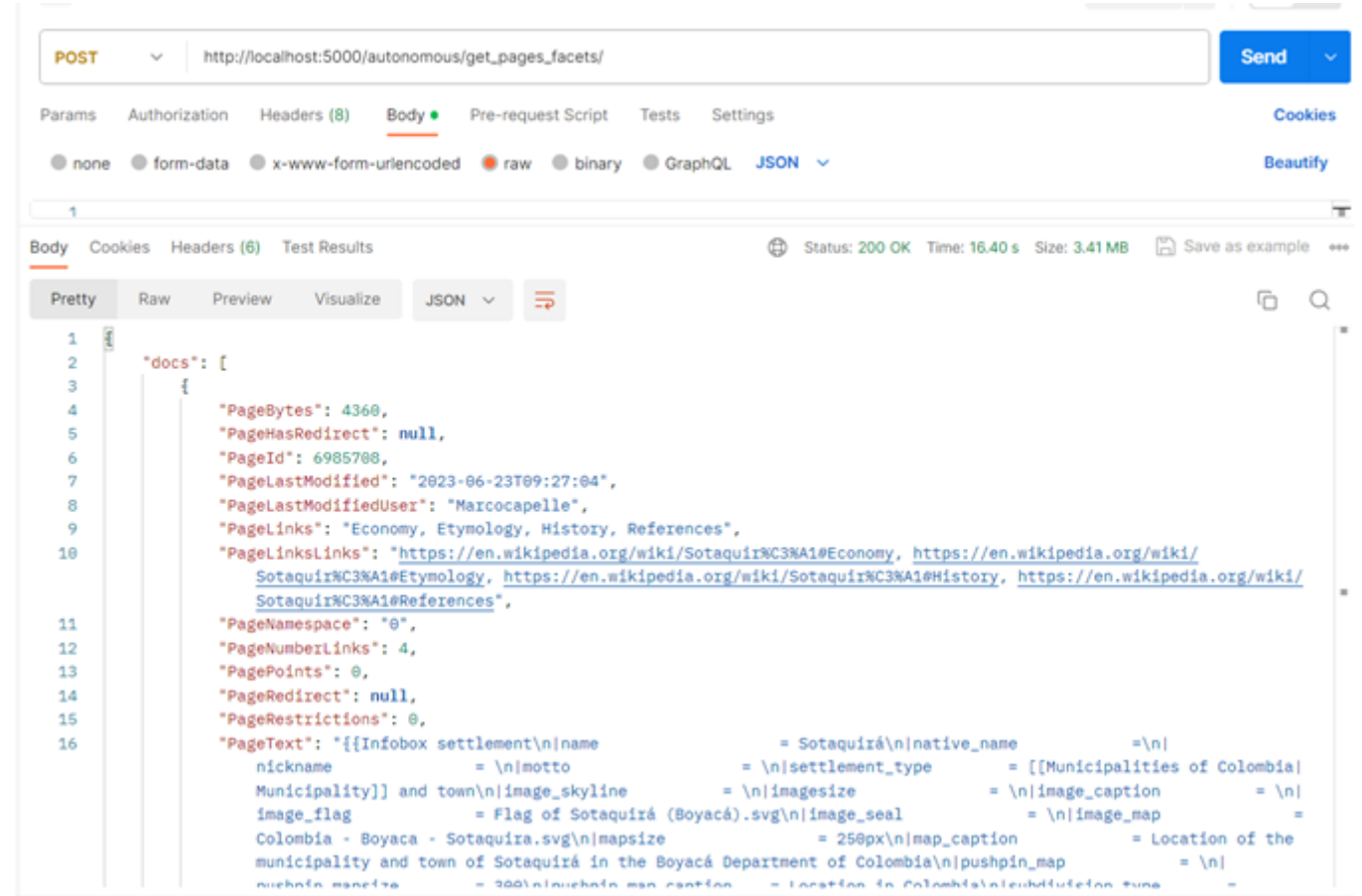
"/autonomous/get_page/<id>"



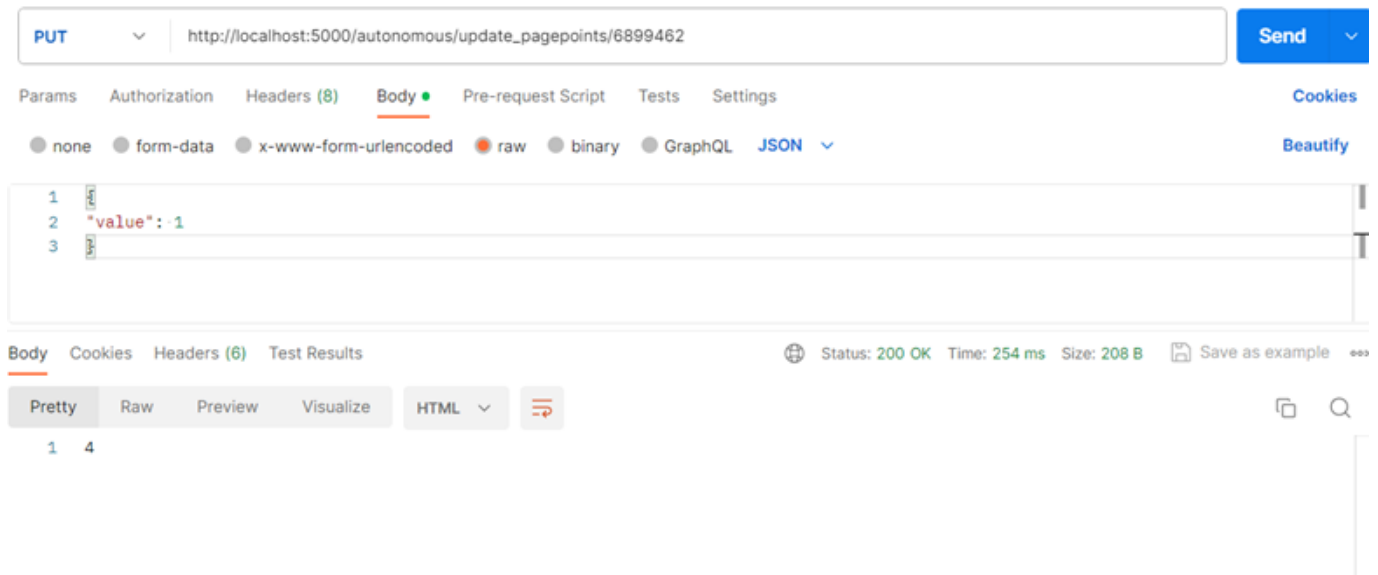
"/autonomous/get_pages/<query>"



"/autonomous/get_pages_facets/"



"/autonomous/update_pagepoints/<pageId>"



UI

El UI es una aplicación en React que se ejecuta en un Docker container en una máquina virtual de Oracle. Nuestra interfaz fue desarrollada con el framework de [NextJS](#). El UI presenta las siguientes funcionalidades:

Login: Mediante un email y password, la persona usuaria puede ingresar a la aplicación, la identidad de la persona es validada contra Firebase.

Register: En caso de que la persona usuaria no cuenta con una cuenta en la aplicación, el usuario podrá crearla contra Firebase. Se solicitan datos como Email, Nombre, Apellidos, Teléfono y Password, una vez que la persona se ha registrado en Firebase, podrá ingresar a la aplicación.

Search: Permite realizar una búsqueda de documentos en el motor de su preferencia (Mongo DB o Autonomous DB), además se le permite al usuario refinar la búsqueda mediante facets (categorías). Cuando la búsqueda es realizada mediante Mongo Atlas se realizan Highlights del texto que ha sido encontrado con el query de búsqueda.

Document: En esta pantalla se muestra toda la información del documento de Wikipedia, este puede ser recuperado de Mongo Atlas o de Autonomous Database, además en esta sección se pueden realizar una votación a favor o en contra por el documento y además se presenta un link directo al documento en el sitio de Wikipedia.

Para conectar el UI con el API, simplemente se realizan requests a los endpoints de Flask del API.

Código

Para la implementación del UI, utilizamos el framework de NextJS, específicamente la versión que utiliza el [Pages Router](#). Esta es la implementación más vieja de NextJS. La versión más nueva utiliza otra metodología llamada [App Router](#). Decidimos utilizar la versión del Pages Router porque esta es más intuitiva. Tiene un sistema de enrutamiento de páginas por medio de un file system, por lo que es muy claro visualizar la estructura de las rutas. También, hay documentación más extensa y detallada sobre esta implementación, por lo que quedaba más sencillo investigar sobre cómo implementar funcionalidades. NextJS se aprovecha de las optimizaciones de React para reducir el procesamiento de la página y realizar lo mayor posible del lado del servidor, con las técnicas de [Static Generation](#) y [Server-Side Rendering](#). Sin embargo, estas técnicas son más

como para páginas estáticas que no cambian tanto con la interacción del usuario, algo que no ocurre en la página de búsqueda de palabras, por ejemplo, por lo que en el proyecto se decidió trabajar más del lado del Client-Side Rendering, donde era el browser quien ejecutaba el código de JavaScript. Next implementa sus [propios componentes de React](#) como `, , ,` etc. Estos aprovechan las optimizaciones de React. Adicionalmente, se utilizó el hook de [useRouter](#) para poder navegar entre páginas al hacer click en un botón o algo similar.

Recomendaciones y Conclusiones

Conclusiones:

1. Las máquinas virtuales y la nube son excelentes opciones para tener una base de datos confiable, sin tener los problemas de mantenerlas y expandirlas.
2. Las aplicaciones encapsuladas en contenedores Docker son altamente portátiles, lo que facilita la implementación en diferentes entornos sin preocuparse por las diferencias de configuración.
3. Con este proyecto, pudimos concluir que para realizar este tipo de procesamiento y de búsquedas, las bases de datos NoSQL son mucho más rápidas que las bases de datos relacionales.
4. Para aumentar el rendimiento de una consulta compleja, se puede usar un materialized view con base en las tablas y campos que tenemos que obtener. Aunque la consulta inicial se retrasa por el procesamiento de la vista, las siguientes son beneficiadas sustancialmente al no tener que repetir las consultas complejas. Por ejemplo, logramos reducir el procesamiento de algunos minutos a unos pocos segundos.
5. Si se desea enviar un request entre el UI y el API en Flask, es posible que dé un CORS Error. Esto es porque se usa la same-origin policy, que solo permite que los requests que se acepten sean del mismo origen, ya sea dominio, esquema o puerto (MDN Contributors, 2023b). Dos URI's tienen el mismo origen si el protocolo, puerto y host son iguales (MDN Contributors, 2023a). El Cross-Origin Resource Sharing es más bien lo contrario. Existe una librería llamada Flask_CORS, que habilita esta política en toda la aplicación (Flask s.f.). De esta manera, los requests empiezan a servir.
6. Un índice de búsqueda permite a MongoDB buscar y recuperar resultados de manera más eficiente cuando se realizan consultas de texto completo. Esto se traduce en tiempos de respuesta más rápidos para las consultas de búsqueda.
7. El pipeline de agregación de MongoDB proporciona una forma flexible y poderosa de realizar operaciones complejas en los datos. Se pueden combinar múltiples etapas de agregación para realizar tareas como filtrado, transformación, agrupación y cálculos avanzados.
8. MongoDB Atlas posee un alto rendimiento y bajo tiempo de respuesta, especialmente cuando se trata de operaciones de lectura y escritura. Si la velocidad de respuesta es una prioridad, MongoDB es una buena opción a considerar.
9. Una base de datos NoSQL es excelente también para almacenar logs. Como cada registro puede tener características diferentes, se pueden almacenar los logs de diferentes componentes o sistemas en la misma tabla. Por ejemplo, en este caso los requests hechos a los endpoints de Mongo y los endpoints de Autonomous son diferentes. No obstante, se pueden almacenar todos en la misma tabla.
10. Oracle destaca en el mundo de las bases de datos al ofrecer una amplia gama de soluciones confiables y escalables. Desde bases de datos relacionales hasta opciones NoSQL, Oracle proporciona herramientas robustas que pueden adaptarse a una variedad de necesidades.
11. Los Full-Text Index ayudan a la optimización de las consultas al permitir búsquedas rápidas y precisas dentro de grandes volúmenes de texto. Facilitan la identificación de palabras y frases específicas, mejorando significativamente el rendimiento y la eficiencia en operaciones relacionadas con contenido textual.

Recomendaciones:

1. Para pasar la información de login entre páginas se pueden usar Query Parameters en el link o meterlas en el LocalStorage y en la siguiente página recuperarlas.
2. Para hacer logs, podemos identificarlos cada uno con la fecha en que se tomaron y un número aleatorio, por si se generan más de un 1 log en exactamente el mismo tiempo.
3. Buscar estrategias para no repetir el procesamiento dos veces. En el loader, utilizamos un documento de bitácora para no tener que recorrer el mismo documento dos veces. En la parte de Autonomous Database, intentamos encontrar una forma para no repetir la misma operación.
4. Estudiar más a fondo las operaciones que se pueden realizar en el pipeline de Mongo ya que esta herramienta es muy fuerte y podría disminuir el tiempo de transformar datos, obtenerlos y agruparlos entre otras funciones.
5. Para un sistema de autenticación de usuarios, Firebase Authentication es una muy buena opción. Es muy segura y posee una REST API en caso de que los drivers o librerías disponibles no tengan las funcionalidades que se desean.
6. Cuando se trabaja en el desarrollo para una máquina virtual, es posible que sea más conveniente programar directamente en la máquina local de uno hasta que ya sirva correctamente y luego subirlo a la máquina virtual. Esto agiliza mucho el desarrollo porque se evita el largo proceso de destruir la máquina virtual, añadir la imagen a Docker Hub, añadir el contenedor a la máquina virtual y volver a montar la máquina virtual. Nada más hay que tener en cuenta que lo que estamos desarrollando sí sea ejecutable por la arquitectura de la máquina virtual.
7. Cuando se trabaje con una base de datos, es recomendable usar una que tenga un driver o librería nativa para el lenguaje con el que estamos trabajando. Si no, hay que procurar que tenga una alternativa, como un REST API. Si no, va a ser muy complicado extraer e insertar datos a la base de datos.
8. Al utilizar un sistema de control de versiones, como Git, considerar implementar una metodología como Git Flow que permite organizar y gestionar eficientemente el flujo de trabajo.
9. Es importante implementar herramientas de monitorización y recolección de métricas para cualquier aplicación ya que esto permite tener visibilidad en tiempo real sobre el rendimiento, la disponibilidad y la salud del sistema.
10. Establecer un estándar para la entrada y salida del contenido del request a un REST API. Nosotros no definimos la estructura al inicio del proyecto, por lo que luego comunicar el UI con el API requirió un mayor grado de comunicación y coordinación que pudo haber sido evitado o reducido habiendo dejado en claro el formato de los requests.

Referencias

Bakker, K. C. (2021, March 11). Python utility function: retry with exponential backoff - <https://keestalkstech.com>. <https://keestalkstech.com/2021/03/python-utility-function-retry-with-exponential-backoff/#without-typings>.

Firebase. (s. f.). Firebase Auth REST API. Firebase.com. Recuperado 10 de octubre de 2023, de <https://firebase.google.com/docs/reference/rest/auth>

Flask-CORS — Flask-Cors 3.0.10 documentation. (s. f.). <https://flask-cors.readthedocs.io/en/latest/> Manage users. (s. f.). Firebase. <https://firebase.google.com/docs/auth/admin/manage-users>

MDN Contributors. (2023a, julio 4). Same-origin policy - Security on the web | MDN. Recuperado 12 de octubre de 2023, de https://developer.mozilla.org/en-US/docs/Web/Security/Same-origin_policy

MDN Contributors. (2023b, agosto 10). Cross-Origin Resource Sharing (CORS) - HTTP | MDN. Recuperado 12 de octubre de 2023, de <https://developer.mozilla.org/en-US/docs/Web/HTTP/CORS>